



Automatización del despliegue y configuración de sistemas Windows mediante Desired State Configuration (DSC)

Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:
Felipe H.C.
Tutor/es:
Félix G.
Félix J.



**Facultad
Informática
Universidad
Murcia**

29 de Mayo de 2026

Automatización del despliegue y configuración de sistemas Windows mediante Desired State Configuration (DSC)

Autor

Felipe H.C.

Tutores

Félix G.

Dpto. de Ingeniería de la Información y las Comunicaciones

Félix J.

Dpto. de Ingeniería y Tecnología de Computadores



Grado en Ingeniería Informática



Murcia, 29 de Mayo de 2026

Preámbulo

La creciente complejidad de los entornos informáticos actuales ha convertido la automatización en un elemento clave para garantizar la eficiencia, la repetibilidad y la fiabilidad en la administración de sistemas. En el ámbito de los sistemas Windows, muchas tareas de despliegue, configuración y mantenimiento continúan realizándose de forma manual, lo que incrementa el tiempo de puesta en marcha, dificulta la homogeneidad entre equipos y aumenta la probabilidad de errores de configuración. Esta situación resulta especialmente relevante en escenarios donde es necesario desplegar de forma consistente varias máquinas con roles diferenciados.

El presente Trabajo Fin de Grado surge de la necesidad de estudiar y aplicar un enfoque automatizado que permita reducir la intervención manual en este tipo de entornos, apoyándose en herramientas y metodologías propias de la automatización de infraestructuras. En concreto, el trabajo se centra en el uso de Desired State Configuration (DSC) como tecnología declarativa para definir y aplicar el estado deseado en sistemas Windows, complementándola con mecanismos de instalación desatendida, virtualización y aprovisionamiento automatizado.

La finalidad principal del estudio es diseñar, implementar y validar una solución capaz de automatizar tanto el despliegue inicial como la configuración específica de varias máquinas virtuales Windows, atendiendo al rol que desempeñan dentro de la infraestructura del laboratorio. Para ello, partimos de una imagen base personalizada de Windows Server, se genera una plantilla reutilizable y se emplean técnicas de automatización para crear y configurar de manera coherente un controlador de dominio, un servidor de ficheros y un cliente de Windows.

El alcance del trabajo comprende desde la preparación del medio de instalación y la creación de plantillas reutilizables hasta la configuración declarativa de los sistemas y la validación de su funcionamiento en un entorno virtualizado. Además, se contempla una fase final de bastionado basada en guías del Centro Criptológico Nacional (CCN), con el objetivo de evaluar la compatibilidad entre la automatización del despliegue y la aplicación posterior de medidas de seguridad. Sin embargo, el estudio se limita a un entorno de laboratorio controlado, por lo que no aborda aspectos como la alta disponibilidad, el escalado a gran volumen o la operación en producción.

Agradecimientos

En primer lugar, me gustaría darles las gracias a mis tutores, Félix G. y Félix J., por darme la oportunidad de realizar mi TFG con ellos y abrirme las puertas del CDL.

A mi familia, gracias por el apoyo constante, la paciencia y la confianza durante todos estos largos años. Obviamente, sin ellos no hubiera podido llegar hasta aquí.

A mis abuelos. Aunque ya no estéis, este trabajo y esta carrera también son vuestros.

Por último, me gustaría agradecer a P.ML, la persona que siempre está ahí. Gracias por escucharme, animarme y acompañarme durante estos años, cuando parecía que la carrera se hacía cuesta arriba y no se veía el final.

Declaración firmada sobre originalidad del trabajo

D. **Felipe H.C.**, con DNI ———, estudiante de la titulación de **Grado en Ingeniería Informática** de la Universidad de Murcia y autor del TF titulado “**Automatización del despliegue y configuración de sistemas Windows mediante Desired State Configuration (DSC)**”.

De acuerdo con el Reglamento por el que se regulan los Trabajos Fin de Grado y de Fin de Máster en la Universidad de Murcia (aprobado C. de Gob. 30-04-2015, modificado 22-04-2016 y 28-09-2018), así como la normativa interna para la oferta, asignación, elaboración y defensa de los Trabajos Fin de Grado y Fin de Máster de las titulaciones impartidas en la Facultad de Informática de la Universidad de Murcia (aprobada en Junta de Facultad 27-11-2015)

DECLARO:

Que el Trabajo Fin de Grado presentado para su evaluación es original y de elaboración personal. Todas las fuentes utilizadas han sido debidamente citadas. Así mismo, declara que no incumple ningún contrato de confidencialidad, ni viola ningún derecho de propiedad intelectual e industrial

Murcia, a 29 de Mayo de 2026

Fdo.: Felipe H.C.
Autor del TF

Resumen

Este Trabajo Fin de Grado desarrolla una solución automatizada para el despliegue, configuración y validación de un entorno Windows virtualizado mediante *Desired State Configuration* (DSC). El trabajo implementado cubre el ciclo completo de construcción del laboratorio: preparación de una imagen base personalizada de Windows Server, generación de una plantilla reutilizable en Proxmox, aprovisionamiento de máquinas virtuales mediante Terraform y configuración declarativa de los nodos desplegados mediante PowerShell DSC.

En primer lugar, se ha creado una ISO personalizada de Windows Server que incorpora un archivo `autounattend.xml`, un script de inicialización `Setup.ps1` y componentes auxiliares necesarios para el entorno, como controladores de virtualización, Cloudbase-Init y módulos DSC. A partir de esta imagen, se despliega una máquina base en Proxmox, se prepara el sistema, se ejecuta Sysprep y se genera una plantilla reutilizable. Esta plantilla permite desplegar nuevos nodos con una base homogénea y reduce la intervención manual en las fases posteriores.

Sobre dicha plantilla se ha definido el aprovisionamiento de la infraestructura mediante Terraform. La configuración creada permite desplegar de forma parametrizada un controlador de dominio, un servidor de ficheros y un cliente Windows, asignando a cada máquina sus recursos, dirección IP, configuración inicial y, en el caso del servidor de ficheros, almacenamiento adicional para datos. Terraform se utiliza exclusivamente para la creación de infraestructura, mientras que la configuración interna de los sistemas se delega en DSC.

La fase principal de configuración se ha realizado mediante PowerShell DSC. En el controlador de dominio se automatiza la instalación de los roles necesarios, la promoción del servidor, la configuración DNS y la creación de una estructura básica de Active Directory con unidades organizativas, usuarios y grupos. En el servidor de ficheros se automatiza la unión al dominio, la preparación del almacenamiento, la creación de recursos compartidos SMB, la activación de *Access-Based Enumeration* y la asignación de permisos NTFS y SMB mediante un modelo basado en grupos. El cliente Windows se incorpora al dominio y se utiliza como nodo de validación funcional del entorno.

Además, el trabajo incorpora mecanismos de continuidad administrativa antes de aplicar medidas de bastionado. Para ello se definen cuentas administrativas alternati-

vas de tipo *BreakGlass*, tanto locales como de dominio, y se controla su pertenencia a los grupos necesarios. Esta decisión permite conservar la capacidad de administración del laboratorio cuando las políticas de seguridad modifican/desactivan cuentas por defecto, reglas de acceso remoto o privilegios administrativos.

La solución se valida mediante pruebas funcionales sobre los componentes desplegados. Se comprueba la disponibilidad del dominio, la resolución DNS, la integración de los nodos en Active Directory, el estado de las configuraciones DSC, el acceso permitido y denegado a recursos compartidos y la administración remota. Posteriormente, se aplican guías de bastionado del CCN-CERT y se repiten las comprobaciones esenciales para verificar que el entorno mantiene su operatividad y sigue siendo administrable tras el endurecimiento.

Los resultados obtenidos muestran que la combinación de una imagen base personalizada, plantillas reutilizables, Terraform y DSC permite construir un entorno Windows reproducible, modular y mantenible. El trabajo demuestra que la automatización no solo permite desplegar máquinas virtuales, sino también configurar servicios de infraestructura, validar su funcionamiento real y preparar el entorno para la aplicación posterior de medidas de seguridad.

Extended Abstract

The administration of Windows-based infrastructures involves many repetitive tasks related to operating system installation, system preparation, role configuration, user and group management, permission assignment and security policy application. When these tasks are performed manually, the resulting process is slow, difficult to reproduce and prone to configuration errors. This becomes especially relevant when several machines with different roles must be deployed in a coordinated way, as small differences introduced during installation or configuration can lead to inconsistent systems and make the environment harder to maintain or rebuild.

This End-of-degree dissertation addresses this problem by designing, implementing and validating an automated solution for the deployment and configuration of Windows systems using Desired State Configuration (DSC). The goal is not limited to automating isolated tasks, but to define a complete workflow that covers the preparation of a base image, the creation of a reusable virtual machine template, the provisioning of infrastructure and the declarative configuration of the deployed systems according to their role. The proposed solution is developed in a controlled laboratory environment and aims to improve reproducibility, modularity and maintainability when compared with a manual deployment process.

The laboratory scenario is composed of three main nodes: a domain controller, a file server and a Windows client. These roles were selected because they represent a realistic and common Windows administration environment. The domain controller provides identity, directory and name resolution services. The file server offers shared resources and applies permissions based on the users and groups defined in the domain. The client is used to validate the behavior of the infrastructure from the perspective of an end system joined to the domain. This combination makes it possible to evaluate not only the deployment of individual machines, but also the interaction between the different components of the environment.

The solution follows a layered architecture in which each tool has a clearly defined responsibility. The first stage consists of preparing a customized Windows Server installation image. Instead of installing each machine manually from a standard ISO, the project creates a modified installation medium that includes an unattended answer file, initialization scripts and auxiliary components required for later stages. This approach reduces manual intervention during the operating system installation and ensures that

the initial system is created under consistent conditions.

The unattended installation is based on an `autounattend.xml` file. This file defines the installation parameters and allows Windows Server to be installed without requiring manual input during the setup process. Its use is important because it turns the installation procedure into a reproducible and documented step. Decisions that would otherwise be introduced through the graphical installer are expressed in a configuration artifact that can be stored, reviewed and reused. This is consistent with the general philosophy of the project: moving operational decisions from manual procedures to versionable files and scripts.

The customized image also includes a `Setup.ps1` script, which is executed after the installation. This script prepares the system for the following phases of the workflow by enabling remote administration mechanisms, arranging auxiliary files and preparing the machine for later configuration with PowerShell and DSC. This separation between the answer file and the initialization script improves maintainability. The answer file is used for installation-specific parameters, while the PowerShell script provides the flexibility required for system preparation tasks that may evolve throughout the project.

Auxiliary components are also included in the base image. These components may include virtualization drivers and tools, Cloudbase-Init files, DSC modules and other resources required in environments with limited access to external repositories. Including them in advance makes the process more robust and reduces the dependency on downloads during deployment. This is particularly useful in laboratory scenarios or controlled environments where it is desirable to know exactly which components are being used.

Once the customized installation medium is available, it is used to deploy an initial virtual machine in Proxmox. This machine is not one of the final nodes of the laboratory. Instead, it is updated, prepared, generalized and converted into a reusable template. The creation of a template is a key design decision because it avoids repeating the complete Windows installation process for every new node. It also ensures that all machines start from a common and controlled base, reducing deployment time and improving consistency between systems.

Before converting the machine into a template, the operating system is updated and prepared for cloning. The generalization process is performed with Sysprep, which removes machine-specific information and leaves the system ready to be cloned into independent virtual machines. In this project, Sysprep is used together with Cloudbase-Init so that new virtual machines created from the template can receive specific parameters during deployment. This allows the template to be reusable while still permitting each node to have its own identity and configuration.

The provisioning of the virtual infrastructure is performed with Terraform. Terraform is used as an Infrastructure as Code tool to define and create the virtual machines in Proxmox. Its role is separated from the configuration tasks performed by DSC. Terraform is responsible for creating the machines, assigning resources, defining disks and network parameters and instantiating nodes from the template. DSC is responsible for configuring the operating system and the services inside each machine. This separation of responsibilities makes the architecture clearer and easier to maintain.

Terraform is involved in two phases of the project. First, it is used to create the initial base virtual machine from the customized ISO. Later, after the template has been generated, it is used again to deploy the final nodes: the domain controller, the file server and the Windows client. This means that Terraform is not only used at the end of the process, but also contributes to the construction of the base resource from which the rest of the environment depends.

After the machines have been provisioned, the configuration phase is carried out using PowerShell Desired State Configuration. The project uses the classic DSC model associated with Windows PowerShell 5.1. This choice is aligned with the Windows-centered nature of the environment, the use of traditional infrastructure roles and the need for mature configuration mechanisms. Although newer versions of DSC exist, the classic model remains suitable for this scenario because it integrates naturally with Windows Server, PowerShell and the Local Configuration Manager.

DSC allows the configuration of each node to be expressed declaratively. Instead of defining only a sequence of commands, the configuration describes the desired state that the system must reach. Resources can be used to ensure that Windows features are installed, services are enabled, folders exist, local groups contain the required members or machines are joined to the domain. This approach improves readability and supports the idea of maintaining configuration as code.

The DSC configuration is organized into common and role-specific parts. Common configurations include settings that are required by several nodes, such as basic system preparation, remote management and shared prerequisites. Role-specific configurations define the particular behavior of each machine. The domain controller configuration installs the required features, promotes the server and creates the basic domain structure, including organizational units, users and groups. This node is essential because the other systems depend on the availability of the domain and DNS services.

The file server configuration prepares the storage structure, creates shared folders and assigns permissions according to the groups defined in the domain. This part of the project demonstrates the usefulness of automation in a practical administration

scenario, since file server configuration involves several elements that can easily become inconsistent if performed manually. The project uses a group-based permission model, avoiding the direct assignment of permissions to individual users. This makes the configuration more realistic and easier to modify in future extensions.

The Windows client is used mainly for validation. It is configured and joined to the domain so that the deployed services can be tested from the point of view of a Windows client. Through the client, it is possible to check domain integration, name resolution, access to shared resources and the general behavior of the environment. Its inclusion is important because it proves that the automated deployment is not limited to creating server-side components, but produces an infrastructure that can actually be used.

A relevant part of the project is the inclusion of a security hardening phase based on CCN-CERT guidelines. This phase is applied after the functional deployment has been completed. Its purpose is not to perform a complete security audit, but to verify whether the automated environment remains operational after security measures are introduced. Hardening policies may affect administrative accounts, remote access, firewall rules or other mechanisms required for system management. Therefore, their impact must be considered as part of the deployment lifecycle.

For this reason, the project includes the preparation of alternative administrative mechanisms before applying hardening measures. In particular, BreakGlass accounts are considered to preserve administrative access if default accounts are disabled, restricted or affected by security policies. These accounts may exist at local or domain level and must be added in a controlled way to the required local or domain groups. In this work, they are not treated as a way to bypass security, but as a continuity measure that prevents the administrator from losing access to the environment after hardening.

Validation is an essential part of the work. The success of the solution cannot be measured only by checking that scripts finish without errors. It must be verified that the resulting systems actually provide the expected functionality. The validation includes checking that the domain is available, DNS resolution works correctly, the file server is joined to the domain, shared resources are accessible according to the defined permissions and the client can interact with the deployed services. After applying the CCN-CERT hardening measures, additional checks are required to ensure that essential services and administrative access remain available.

The validation phase confirmed that the automated workflow was able to produce a functional Windows laboratory environment composed of a domain controller, a file server and a Windows client. The resulting infrastructure satisfied the validation criteria defined in the methodology: the domain controller provided Active Directory Domain Services and DNS resolution, the file server was integrated into the domain,

the client was able to authenticate against the domain, and shared resources were accessed according to the group-based permission model.

After the application of CCN-CERT hardening guidelines, the essential services of the environment were checked again in order to verify operational continuity. This final validation made it possible to assess not only whether the deployment was reproducible, but also whether the environment remained administrable after security restrictions were introduced.

The project also shows that DSC remains a useful technology for Windows configuration automation. Its declarative model, its integration with PowerShell and its ability to manage Windows roles and resources make it appropriate for this type of scenario. At the same time, Terraform provides a clear and reproducible mechanism for the infrastructure layer. The combination of both tools allows each one to be used for the task for which it is best suited.

The work has some limitations. The environment is a laboratory and does not reproduce all the requirements of a production infrastructure, such as high availability, disaster recovery, centralized monitoring or large-scale deployment. The number of roles is also limited to those required to demonstrate the approach: a domain controller, a file server and a client. In addition, the hardening phase is treated as an operational compatibility check rather than a complete cybersecurity audit.

Future work could extend this foundation by adding new Windows roles, more detailed automated tests, stronger integration of the hardening process and a comparison with other orchestration approaches such as Ansible or newer DSC versions. These extensions would make it possible to evaluate the scalability of the proposed workflow and its applicability to more complex scenarios. They would also help transform the laboratory into a reusable teaching or demonstration environment for system administration, virtualization and Windows security practices.

In conclusion, this End-of-degree dissertation demonstrates that the deployment and configuration of Windows systems can be automated in a coherent and reproducible way by combining unattended installation, reusable templates, Infrastructure as Code and declarative configuration. The solution reduces manual intervention, improves consistency between machines and provides a structured basis for future extensions. It also highlights that automation should not be limited to making deployment faster, but should also consider security, validation and administrative continuity throughout the lifecycle of the infrastructure.

Índice general

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Problema abordado y entorno de referencia	2
1.3. Objetivos del trabajo	7
1.4. Enfoque metodológico	9
1.5. Estructura del documento	9
2. Estado del arte	11
2.1. Automatización de sistemas e Infraestructura como Código	11
2.2. Terraform como herramienta de aprovisionamiento	12
2.3. Desired State Configuration: arquitectura, funcionamiento y evolución .	14
2.4. Ansible y automatización de sistemas Windows	18
2.5. Comparativa tecnológica y justificación de la solución adoptada	19
2.6. Resumen del enfoque seleccionado	21
3. Metodología y Requisitos del Sistema	23
3.1. Metodología de desarrollo	23
3.2. Alcance y limitaciones	24
3.3. Requisitos del sistema y del entorno	27
3.4. Entorno de trabajo y herramientas empleadas	28
3.5. Criterios de validación	30
4. Diseño, implementación, validación y análisis de resultados	33
4.1. Diseño de la solución	33
4.1.1. Arquitectura general y separación por capas	34
4.1.2. Decisiones principales de diseño	35
4.1.3. Diseño de la imagen base y de la plantilla reutilizable	36
4.1.4. Diseño del aprovisionamiento y parametrización inicial	36
4.1.5. Diseño de la configuración con DSC	37
4.1.6. Diseño de la validación y del bastionado	37
4.2. Implementación del flujo automatizado	38
4.2.1. Preparación de la imagen base y creación de la plantilla	38
4.2.2. Aprovisionamiento con Terraform y Cloudbase-Init	41
4.2.3. Configuración de los nodos mediante PowerShell DSC	42
4.2.4. Aplicación automatizada de guías CCN-STIC	45

4.3.	Validación funcional y comprobación tras bastionado	46
4.3.1.	Escenario de validación	46
4.3.2.	Caso de uso de acceso a recursos compartidos	46
4.3.3.	Resultados antes del bastionado	48
4.3.4.	Resultados después del bastionado	48
4.4.	Discusión y análisis de resultados	49
4.4.1.	Análisis del enfoque por capas	49
4.4.2.	Análisis de la validación funcional	50
4.4.3.	Análisis del bastionado y de la continuidad administrativa . . .	51
4.4.4.	Incidencias técnicas y decisiones adoptadas	51
4.4.5.	Valoración global de los resultados	52
5.	Conclusiones y vías futuras	55
5.1.	Conclusiones	55
5.2.	Limitaciones del trabajo	57
5.3.	Vías futuras	58
	Bibliografía	59
	Lista de Acrónimos y Abreviaturas	63
A.	Código fuente y configuraciones de automatización	65
A.1.	Preparación de la imagen base	65
A.1.1.	Generación de la ISO personalizada	65
A.1.2.	Estructura funcional del script Setup.ps1	66
A.2.	Creación de la plantilla reutilizable	67
A.2.1.	Configuración base con 01-Base.ps1	67
A.2.2.	Generalización con Sysprep y Cloudbase-Init	68
A.3.	Aprovisionamiento con Terraform y Cloudbase-Init	69
A.3.1.	Definición parametrizada de nodos (vars.tf)	69
A.3.2.	Clonado de máquinas desde la plantilla main.tf	70
A.3.3.	Disco adicional para el servidor de ficheros (main.tf)	71
A.4.	Configuración DSC del controlador de dominio	71
A.4.1.	Promoción del controlador de dominio	71
A.4.2.	Creación de objetos de Active Directory	72
A.5.	Configuración DSC del servidor de ficheros	73
A.5.1.	Unión del servidor de ficheros al dominio	73
A.5.2.	Rol de ficheros, recurso SMB y Access-Based Enumeration . . .	74
A.5.3.	Modelo de permisos NTFS y SMB	76
A.6.	Configuración DSC del cliente Windows	77
A.6.1.	Preparación y unión del cliente al dominio	77
A.7.	Aplicación de guías CCN-STIC	79
A.7.1.	Copia de paquetes y scripts al controlador de dominio	79

A.7.2. Configuración DSC para aplicar guías CCN-STIC	79
A.7.3. Ejecución de la aplicación de guías	81
A.8. Comandos de validación funcional	81
A.8.1. Comprobación del dominio y DNS	81
A.8.2. Comprobación de unión al dominio	82
A.8.3. Comprobación del estado de DSC	82
A.8.4. Comprobación de recursos compartidos	83
A.8.5. Comprobación de acceso desde cliente	83
A.8.6. Comprobación de administración remota	83
A.9. Observaciones sobre credenciales y seguridad	84

Índice de figuras

2.1.	Arquitectura simplificada de PowerShell DSC en el modelo clásico.	15
2.2.	Comparación entre los modelos Push y Pull en PowerShell DSC. .	16
3.1.	Flujo de generación de la plantilla y despliegue automatizado del entorno.	23
3.2.	Entorno de laboratorio virtualizado y flujo de despliegue.	30
4.1.	Escenario de validación funcional antes y después del bastionado. .	47

Índice de tablas

1.1.	Estimación del esfuerzo y riesgos asociados al despliegue manual del entorno.	3
1.2.	Impacto operativo del bastionado y mitigación mediante cuentas BreakGlass.	6
2.1.	Comparativa entre DSC 1.1, DSC 2.0 y Microsoft DSC 3.0	17
2.2.	Comparativa de tecnologías de automatización consideradas	20
3.1.	Alcance funcional de la solución implementada	25
3.2.	Limitaciones del trabajo	26
3.3.	Requisitos funcionales del entorno	27
3.4.	Requisitos no funcionales del entorno	28
3.5.	Herramientas empleadas en el proyecto	29
3.6.	Nodos principales del laboratorio	29
3.7.	Criterios de validación del proyecto	31
3.8.	Pruebas funcionales previstas	31
4.1.	Capas principales de la solución propuesta	35
4.2.	Decisiones principales de diseño de la solución	35
4.3.	Elementos incorporados en la imagen base	40
4.4.	Proceso de creación de la plantilla base reutilizable	41
4.5.	Parámetros aplicados por Terraform durante el aprovisionamiento	42
4.6.	Bloques principales de configuración DSC	43
4.7.	Guías CCN-STIC utilizadas en el laboratorio	45
4.8.	Escenario funcional de validación basado en perfiles de usuario . .	47
4.9.	Resultados de validación funcional antes del bastionado	48
4.10.	Resultados de validación después del bastionado	49
4.11.	Incidencias técnicas y decisiones de diseño adoptadas	51
5.1.	Relación entre objetivos específicos y evidencias obtenidas	55

Índice de Códigos

A.1. Generación de la ISO personalizada con oscdimg	65
A.2. Estructura funcional del script Setup.ps1	66
A.3. Actualización de la VM base	67
A.4. Generalización de la máquina base con Sysprep y Cloudbase-Init	68
A.5. Definición parametrizada de los nodos del laboratorio	69
A.6. Clonado de nodos desde la plantilla base mediante Terraform	70
A.7. Asignación condicional de disco adicional al servidor de ficheros	71
A.8. Fragmento DSC para promoción del controlador de dominio	71
A.9. Fragmento DSC para objetos de Active Directory	72
A.10.Fragmento DSC para unión del servidor de ficheros al dominio	73
A.11.Fragmento DSC para rol de ficheros, recurso SMB y ABE	74
A.12.Estructura representativa para permisos NTFS y SMB	76
A.13.Fragmento DSC para preparación y unión del cliente Windows	77
A.14.Copia de paquetes CCN-STIC y scripts auxiliares al controlador de do- minio	79
A.15.Fragmento DSC para aplicar guías CCN-STIC de forma controlada . .	80
A.16.Ejecución del script de aplicación de guías CCN-STIC	81
A.17.Comprobación del dominio y resolución DNS	81
A.18.Comprobación de unión de nodos al dominio	82
A.19.Comprobación del estado de DSC	82
A.20.Comprobación de recursos SMB y permisos	83
A.21.Comprobación de acceso a recursos desde cliente Windows	83
A.22.Comprobación de administración remota	83

1. Introducción

1.1. Contexto y motivación

La administración de sistemas informáticos se ha vuelto cada vez más compleja debido al aumento del número de servicios, equipos, configuraciones y dependencias que forman parte de una infraestructura. En este contexto, los servidores, estaciones de trabajo, servicios de red, mecanismos de autenticación y recursos compartidos deben mantenerse configurados de forma coherente para garantizar que el entorno funcione correctamente. En infraestructuras Windows, servicios como Active Directory Domain Services permiten organizar objetos de red y facilitar su acceso a usuarios y administradores, mientras que protocolos como SMB proporcionan mecanismos de compartición de archivos entre sistemas Windows [1, 2].

Cuando estas tareas se realizan de forma manual, resulta más difícil asegurar que todos los sistemas se han preparado bajo los mismos criterios y que el proceso pueda repetirse con resultados equivalentes. Esta problemática es especialmente relevante en entornos donde coexisten varios roles interdependientes, ya que la configuración de un sistema puede condicionar el funcionamiento del resto. Por ejemplo, la preparación de un controlador de dominio requiere instalar roles específicos, configurar servicios de resolución de nombres, crear una estructura de directorio y definir usuarios o grupos. Del mismo modo, un servidor de ficheros necesita integrarse en el dominio, preparar almacenamiento, publicar recursos compartidos y aplicar permisos de acceso. Si cada una de estas acciones se ejecuta manualmente, aumentan las posibilidades de introducir diferencias entre despliegues, omitir pasos o depender en exceso de la experiencia de la persona administradora.

Frente a esta situación, la automatización de sistemas surge como respuesta a este tipo de problemas. Su objetivo no es únicamente reducir el tiempo de ejecución de tareas repetitivas, sino también convertir los procedimientos de despliegue y configuración en artefactos documentados, revisables y reutilizables. En este sentido, enfoques como la Infraestructura como Código permiten describir recursos de infraestructura mediante ficheros versionables, reduciendo la dependencia de operaciones manuales sobre interfaces gráficas o consolas administrativas [3, 4]. Este planteamiento se relaciona además con prácticas de entrega y despliegue automatizado, donde la repetibilidad y la reducción de errores manuales son aspectos fundamentales para construir procesos técnicos más fiables [5].

Además del despliegue inicial, la administración de sistemas debe considerar la evolución y el mantenimiento posterior de la configuración. La gestión de configuración permite establecer, controlar y verificar el estado de los sistemas a lo largo del tiempo, evitando que cambios no controlados generen desviaciones respecto al estado previsto. Desde una perspectiva de seguridad, esta gestión resulta especialmente importante, ya que las configuraciones del sistema forman parte de la superficie de exposición y deben mantenerse de acuerdo con criterios definidos [6]. En sistemas Windows, este planteamiento se refleja también en el uso de líneas base de seguridad, que agrupan configuraciones recomendadas para facilitar la aplicación coherente de medidas de protección [7].

La automatización también facilita la validación del entorno resultante. En lugar de comprobar únicamente que una máquina se ha creado o que un script ha terminado sin errores, es posible definir pruebas funcionales que verifiquen si los servicios desplegados cumplen su finalidad: resolución de nombres, autenticación, unión al dominio, acceso a recursos compartidos o aplicación correcta de permisos. Esta capacidad de repetir el despliegue y contrastar su resultado, resulta especialmente útil en entornos de laboratorio, donde es habitual reconstruir sistemas, modificar configuraciones y evaluar distintas decisiones técnicas.

La motivación de este Trabajo Fin de Grado parte de esa necesidad de transformar un proceso manual de despliegue y configuración de sistemas Windows en un procedimiento más reproducible, estructurado y validable. El interés principal no se limita a instalar máquinas virtuales, sino a estudiar cómo puede organizarse un flujo completo de preparación, aprovisionamiento, configuración y comprobación de un entorno Windows con varios roles relacionados entre sí.

1.2. Problema abordado y entorno de referencia

El problema central de este trabajo radica en la ineficiencia, la falta de reproducibilidad y los riesgos de seguridad derivados del despliegue y la configuración manual de infraestructuras basadas en sistemas operativos Windows dentro de entornos virtualizados. Esta problemática presenta dos enfoques críticos fuertemente interconectados: por un lado, la insostenibilidad temporal y la propensión al error humano del aprovisionamiento tradicional; por otro, la necesidad de endurecer los sistemas sin comprometer la continuidad operativa ni la capacidad de administración posterior.

El entorno de referencia utilizado para abordar este problema, está compuesto por tres nodos desplegados en una red de laboratorio: un controlador de dominio (DC), un servidor de ficheros (FS) y una estación de trabajo Windows cliente. El DC proporciona

los servicios de Active Directory y DNS; el FS se integra en el dominio y publica recursos compartidos mediante SMB con permisos basados en grupos; y el cliente Windows permite validar el funcionamiento del entorno desde la perspectiva de un equipo final unido al dominio. Esta combinación permite evaluar no solo el despliegue individual de máquinas virtuales, sino también la interacción entre identidad, resolución de nombres, recursos compartidos, autenticación y control de acceso.

Ineficiencia y métricas estimadas del despliegue manual

En la administración de sistemas tradicional, la preparación de una infraestructura con roles jerárquicos y funcionales diferenciados exige una inversión temporal considerable y una interacción constante con interfaces gráficas de usuario (GUI). Tomando como referencia el escenario base de este TFG, comprendido por — DC,FS y cliente Windows—, un despliegue manual e independiente de la automatización implicaría una secuencia estructurada de tareas cuyos tiempos estimados se desglosan en la Tabla 1.1.

La medición de estos tiempos no pretende constituir una medición experimental exhaustiva, sino una estimación orientativa basada en las tareas necesarias para construir el laboratorio. Su finalidad es mostrar la magnitud del esfuerzo manual requerido y justificar la conveniencia de automatizar un proceso compuesto por múltiples fases repetitivas e interdependientes.

Tabla 1.1: Estimación del esfuerzo y riesgos asociados al despliegue manual del entorno.

Fase / Rol	Tareas principales	Tiempo estimado	Vector de riesgo manual
Instalación del sistema operativo	Instalación interactiva, particionado, configuración inicial, contraseña local y primer arranque.	60–90 min	Diferencias entre máquinas base, parámetros omitidos o configuraciones iniciales no homogéneas.
Controlador de dominio	Instalación de AD DS y DNS, promoción a controlador de dominio, creación del bosque y reinicios asociados.	20–25 min	Errores en la configuración DNS, nombre del dominio, promoción del servidor o estructura inicial.
Servidor de ficheros	Preparación de almacenamiento, creación de carpetas, recursos compartidos SMB y permisos NTFS.	30–45 min	Incoherencias entre permisos NTFS, permisos SMB, herencia de carpetas y grupos de dominio.
Cliente Windows	Configuración de red y DNS, unión al dominio, reinicio y comprobación inicial.	15–20 min	Fallos de unión al dominio, resolución incorrecta o validación incompleta desde el equipo final.
Métrica global	Entorno mínimo de tres nodos operativo.	125–180 min	<i>Configuration drift</i> y pérdida de uniformidad entre despliegues.

En términos de tiempo, este entorno mínimo requiere entre dos y tres horas de dedicación manual acumulada. Si este proceso se replica para múltiples laboratorios el coste temporal escala de forma prácticamente lineal. La automatización, por tanto, no solo reduce el tiempo de intervención, sino que evita que cada reconstrucción del laboratorio dependa de repetir manualmente la misma secuencia de operaciones.

Más allá del coste temporal, el problema principal también se encuentra en la tasa de error humano y en la aparición de *Configuration Drift*, es decir, desviaciones entre la configuración prevista y la configuración realmente aplicada. En un despliegue manual, pequeñas diferencias pueden alterar el funcionamiento global del entorno: una dirección DNS incorrecta puede impedir la unión de equipos al dominio; una unidad organizativa mal definida puede ubicar objetos en una ruta no prevista; una pertenencia errónea a grupos puede conceder o denegar acceso de forma indebida; y una incoherencia entre permisos NTFS y SMB puede provocar que el servidor de ficheros no se comporte como se esperaba.

Vulnerabilidad del estado inicial y justificación del bastionado

Un segundo problema lo encontramos en la exposición de los sistemas una vez desplegados. Un entorno Windows recién instalado puede ser funcional desde el punto de vista operativo, pero no necesariamente adecuado desde el punto de vista de la seguridad. Antes de considerarlo válido, es necesario revisar servicios habilitados, cuentas administrativas, reglas de cortafuegos, políticas locales, protocolos de red, mecanismos de administración remota y permisos efectivos.

En este contexto, el bastionado o endurecimiento técnico resulta necesario para reducir configuraciones innecesarias, limitar la exposición de servicios y aplicar políticas de seguridad coherentes. En sistemas Windows, esta revisión de seguridad puede incluir la deshabilitación de componentes heredados, la restricción de mecanismos de resolución de nombres no necesarios, la limitación de accesos administrativos, el ajuste de reglas de cortafuegos y la aplicación de directivas de seguridad alineadas con guías o líneas base reconocidas [6, 7].

Conviene señalar que no todos los componentes heredados se encuentran habilitados por defecto en versiones modernas de Windows Server. Por ejemplo, SMBv1 se encuentra en desuso y no se instala por defecto en Windows Server 2019 y versiones posteriores. Sin embargo, su comprobación es importante dentro de una fase de bastionado, especialmente cuando se trabaja con plantillas, imágenes personalizadas, compatibilidad heredada o entornos donde pueden haberse incorporado componentes adicionales [8].

Asimismo, mecanismos como LLMNR (*Link-Local Multicast Name Resolution*) o NBT-NS (*NetBIOS Name Service*) deben revisarse dentro del proceso de endurecimiento cuando no sean necesarios para el funcionamiento del entorno. Si permanecen activos en determinados escenarios, pueden facilitar ataques de envenenamiento de nombres en la red local, captura de credenciales o movimientos laterales. Por ello, el problema no se limita a desplegar sistemas funcionales, sino a comprobar que el estado final del laboratorio reduce la exposición innecesaria sin romper los servicios que sí deben permanecer operativos.

En el marco de este TFG, la necesidad de bastionado se relaciona también con el Esquema Nacional de Seguridad, regulado por el Real Decreto 311/2022, y con las guías de seguridad del CCN-CERT empleadas como referencia técnica [9, 10]. El objetivo no es realizar una auditoría completa de cumplimiento, sino comprobar que un entorno Windows automatizado puede someterse a una fase posterior de endurecimiento sin perder funcionalidad esencial.

Conflicto operativo del bastionado: guías CCN-STIC y continuidad administrativa

La aplicación de medidas de seguridad rigurosas introduce un conflicto operativo importante si se realiza de forma aislada. Cuanto más restrictivas son las políticas aplicadas, mayor es el riesgo de afectar a mecanismos habituales de administración, cuentas por defecto, acceso remoto, pertenencia a grupos locales o servicios necesarios para gestionar el entorno.

En este trabajo se aplican configuraciones de bastionado basadas en guías CCN-STIC concretas: la CCN-STIC-570A25 para sistemas Windows Server, la CCN-STIC-573-25 para el servidor de ficheros y la CCN-STIC-599AB23 para el cliente Windows. Estas guías permiten introducir restricciones de seguridad sobre los distintos tipos de nodo del laboratorio y comprobar su impacto sobre la infraestructura previamente desplegada.

Una de las medidas de bastionado más sensibles desde el punto de vista operativo consiste en la deshabilitación o restricción de cuentas integradas como **Administrador** e **Invitado**. Estas cuentas son especialmente relevantes porque existen por defecto en sistemas Windows y pueden convertirse en objetivo de ataques automatizados de fuerza bruta, *password spraying* o técnicas de movimiento lateral. Sin embargo, si estas directivas se despliegan de forma masiva mediante Directivas de Grupo sin una planificación previa, los administradores legítimos pueden sufrir una pérdida de acceso administrativo habitual, especialmente si no existen cuentas alternativas previamente preparadas. En ese escenario, la gestión remota o local de los servidores puede quedar

limitada, dificultando la recuperación y la operación normal de la infraestructura.

La Tabla 1.2 resume el impacto técnico de algunas medidas de bastionado consideradas en el proyecto y el mecanismo de mitigación previsto mediante cuentas de contingencia.

Tabla 1.2: Impacto operativo del bastionado y mitigación mediante cuentas BreakGlass.

Ámbito	Medida de bastionado considerada	Impacto operativo posible	Mecanismo de mitigación
Cuentas por defecto	Deshabilitación o restricción de la cuenta Administrador integrada.	Pérdida de acceso si no existen cuentas administrativas alternativas.	Creación automatizada de cuentas de emergencia <i>BreakGlass</i> .
Administración remota	Restricción de accesos remotos, reglas de cortafuegos o políticas locales.	Bloqueo de WinRM, RDP u otros mecanismos de gestión.	Definición previa de excepciones y permisos controlados.
Privilegios administrativos	Limitación de pertenencia a grupos locales o de dominio.	Imposibilidad de realizar tareas de soporte o recuperación.	Inclusión controlada de cuentas BG en los grupos estrictamente necesarios.
Servidor de ficheros	Aplicación de políticas de seguridad sobre servicios, permisos y recursos.	Alteración del acceso a recursos compartidos o de la administración del nodo.	Validación funcional antes y después del bastionado.

Para resolver este conflicto entre seguridad y operatividad, el problema abordado en este trabajo se extiende al diseño e integración de mecanismos de continuidad administrativa dentro del propio flujo automatizado. Esto exige la creación planificada de cuentas de emergencia o *BreakGlass*. Estas cuentas no se plantean como un mecanismo para eludir las políticas de seguridad, sino como una vía controlada de administración ante escenarios de contingencia, especialmente cuando las cuentas por defecto quedan deshabilitadas o restringidas.

El desafío técnico no consiste únicamente en crear una cuenta adicional, sino en definir correctamente sus permisos. En este TFG, los usuarios *BreakGlass* se generan de forma automatizada y se incorporan a grupos de seguridad concretos, evitando depender de asignaciones manuales posteriores. De este modo, la continuidad administrativa queda incluida dentro del diseño del entorno y no como una corrección improvisada tras aplicar el bastionado.

En particular, el flujo de automatización contempla:

- La creación de cuentas administrativas alternativas de dominio para conservar la capacidad de gestión centralizada cuando Active Directory se encuentra operativo.
- La creación o conservación de cuentas administrativas locales de emergencia en los nodos, útiles cuando la comunicación con el dominio no está disponible.
- La incorporación controlada de estas cuentas o grupos a los grupos administrativos necesarios, como `Domain Admins` en el dominio o `Administradores y Usuarios de Escritorio Remoto` en los nodos donde sea necesario.
- La validación posterior de que dichas cuentas permiten administrar el entorno tras aplicar las políticas de bastionado.

Esta asignación se realiza siguiendo un criterio de permisos controlados: las cuentas *BreakGlass* deben tener los privilegios necesarios para recuperar o gestionar el entorno, pero no deben utilizarse como cuentas ordinarias de administración diaria. En un entorno de producción sería necesario complementar este modelo con auditoría, rotación de credenciales, custodia segura y procedimientos formales de uso. En el contexto de este TFG, su finalidad es demostrar que la automatización del despliegue debe contemplar también la continuidad administrativa tras el endurecimiento.

De este modo, cuando las políticas de bastionado deshabilitan o restringen la cuenta Administrador integrada, la infraestructura puede seguir siendo accesible y gestionable mediante un canal secundario previamente definido. Por tanto, el problema abordado en este Trabajo Fin de Grado no se limita a ejecutar scripts de instalación o clonar máquinas virtuales. Consiste en diseñar, implementar y validar un flujo de trabajo unificado, reproducible y estructurado por capas, capaz de aprovisionar la infraestructura virtual, configurar los roles funcionales del laboratorio, aplicar medidas de bastionado basadas en las guías CCN-STIC y mantener la continuidad operativa sin depender de intervención manual correctiva.

1.3. Objetivos del trabajo

El objetivo general de este Trabajo Fin de Grado es diseñar, implementar y validar un flujo automatizado para el despliegue, configuración y endurecimiento de un entorno Windows virtualizado mediante *Desired State Configuration* (DSC), integrando instalación desatendida, plantillas reutilizables, aprovisionamiento de infraestructura y validación funcional.

Este objetivo general responde al problema descrito en el apartado anterior: construir una infraestructura Windows compuesta por varios nodos con roles diferenciados,

reduciendo la intervención manual, evitando desviaciones de configuración y manteniendo la capacidad de administración tras aplicar medidas de bastionado. Por tanto, el trabajo no se limita a crear máquinas virtuales, sino que busca automatizar el ciclo completo de preparación, despliegue, configuración, validación y endurecimiento del laboratorio.

La elección de DSC como tecnología principal de configuración se debe a su integración directa con el ecosistema Windows y PowerShell, así como a su capacidad para describir de forma declarativa el estado esperado de un sistema. Aunque existen alternativas como Ansible, Puppet, Chef o Salt, el enfoque de este trabajo se centra específicamente en sistemas Windows, roles clásicos de infraestructura y configuración mediante tecnologías nativas o estrechamente vinculadas al propio sistema operativo. La comparación con otras alternativas se desarrolla con mayor profundidad en el Capítulo 2.

A partir del objetivo general, se definen los siguientes objetivos específicos:

- OE-01** Diseñar una imagen base automatizada de Windows Server y generar una plantilla reutilizable en Proxmox que permita desplegar nuevos nodos de forma homogénea.
- OE-02** Automatizar el aprovisionamiento de la infraestructura virtual mediante Terraform, definiendo los recursos, parámetros de red, almacenamiento y máquinas necesarias para el laboratorio.
- OE-03** Definir y aplicar configuraciones declarativas mediante PowerShell DSC para transformar los nodos desplegados en un controlador de dominio, un servidor de ficheros y un cliente Windows integrado en el dominio.
- OE-04** Incorporar una fase de bastionado basada en guías CCN-STIC, contemplando mecanismos de continuidad administrativa mediante cuentas *BreakGlass* y permisos controlados.
- OE-05** Validar el funcionamiento del entorno antes y después del bastionado, comprobando dominio, resolución DNS, integración de nodos, acceso a recursos compartidos, control de permisos y capacidad de administración.

Estos objetivos permiten estructurar el desarrollo del trabajo en fases claramente diferenciadas. El objetivo **OE-01** se relaciona con la preparación de la base del entorno; el objetivo **OE-02**, con la creación reproducible de la infraestructura; el objetivo **OE-03**, con la configuración funcional de los sistemas; el objetivo **OE-04**, con la seguridad y la continuidad operativa; y el objetivo **OE-05**, con la comprobación experimental de los resultados obtenidos.

1.4. Enfoque metodológico

Para cumplir los objetivos planteados y resolver el problema descrito, en este trabajo se sigue un enfoque metodológico estructurado, propio de la ingeniería de sistemas. El proyecto se organiza mediante un ciclo de desarrollo incremental e iterativo dividido en fases lógicas claramente diferenciadas.

Este flujo de trabajo guía el proyecto paso a paso: comienza con la preparación de la instalación desatendida y la generación de una plantilla reutilizable en el hipervisor; continúa con el aprovisionamiento automático de las máquinas virtuales mediante infraestructura como código; seguidamente, avanza con la configuración declarativa de los diferentes roles mediante DSC; y termina con el bastionado mediante las guías CCN-STIC junto con la validación final del entorno. Cada una de estas etapas está conectada directamente con los objetivos específicos (OE-01 a OE-05) descritos en el apartado anterior, asegurando que cada paso técnico responde a una finalidad concreta del proyecto.

El detalle de esta metodología de desarrollo, el alcance exacto del proyecto, los requisitos técnicos del sistema y la batería de pruebas utilizadas para comprobar que todo funciona correctamente se explican a fondo en el Capítulo 3 de este documento.

1.5. Estructura del documento

El documento se organiza en cinco capítulos principales. Tras esta introducción, el segundo capítulo presenta el estado del arte, revisando los fundamentos de la automatización de sistemas, la Infraestructura como Código (IaC), la evolución de DSC, su comparación con Ansible y la justificación de la solución adoptada.

El tercer capítulo expone la metodología seguida, los requisitos y el entorno de trabajo. El cuarto capítulo describe el diseño, la implementación y la validación de la solución, incluyendo la preparación de la imagen base, la creación de la plantilla, el aprovisionamiento con Terraform, la configuración mediante DSC, la gestión administrativa, la validación funcional y la aplicación de guías CCN-STIC. Finalmente, el quinto capítulo recoge las conclusiones, limitaciones y posibles líneas de trabajo futuro.

2. Estado del arte

2.1. Automatización de sistemas e Infraestructura como Código

La automatización de sistemas surge como respuesta a la necesidad de gestionar infraestructuras cada vez más dinámicas, heterogéneas y difíciles de mantener mediante procedimientos manuales. En un entorno tradicional, la creación de servidores, la configuración de servicios, la asignación de permisos o la aplicación de políticas de seguridad suelen depender de acciones realizadas de forma interactiva por una persona administradora. Este modelo puede ser suficiente en escenarios pequeños o puntuales, pero presenta limitaciones claras cuando el entorno debe reproducirse, auditarse o modificarse de forma controlada.

En este contexto aparece el concepto de Infraestructura como Código, o *Infrastructure as Code* (IaC). Este enfoque propone describir la infraestructura mediante ficheros de configuración versionables, reutilizables y revisables, en lugar de depender exclusivamente de acciones manuales sobre interfaces gráficas o consolas administrativas. De este modo, los elementos de infraestructura dejan de ser configuraciones dispersas y pasan a convertirse en artefactos técnicos que pueden almacenarse, comparar cambios, documentarse y ejecutarse de forma repetible [3, 4].

La literatura técnica distingue habitualmente entre dos grandes grupos de herramientas dentro del ecosistema de automatización. Por un lado, se encuentran las herramientas de aprovisionamiento, orientadas a crear y gestionar recursos de infraestructura como máquinas virtuales, redes, discos, balanceadores o servicios cloud. En este grupo se sitúan soluciones como Terraform, AWS CloudFormation, Azure Bicep o Pulumi. Por otro lado, se encuentran las herramientas de gestión de configuración, cuyo objetivo principal es actuar sobre el sistema operativo y los servicios instalados para llevarlos a un estado deseado. En este grupo aparecen tecnologías como Desired State Configuration, Ansible, Puppet, Chef o Salt.

Esta distinción entre herramientas es importante, ya que el despliegue de un entorno completo de Windows no puede resolverse únicamente creando máquinas virtuales. También es necesario configurar el sistema operativo, instalar roles, unir equipos al dominio, crear usuarios y grupos, preparar recursos compartidos y aplicar permisos.

Por tanto, el problema requiere combinar una capa de aprovisionamiento de infraestructura con una capa de configuración del sistema.

Un aspecto clave de estas tecnologías es su orientación declarativa. A diferencia de los scripts imperativos tradicionales, donde se define una secuencia concreta de comandos que deben ejecutarse en un orden determinado, las herramientas declarativas permiten describir el estado final que debe alcanzar el sistema. A partir de esa definición, la propia herramienta compara el estado real con el estado esperado y determina qué acciones son necesarias para aproximar el entorno a la configuración declarada.

Esta forma de trabajo se relaciona directamente con el principio de idempotencia. Una configuración idempotente puede aplicarse varias veces sobre un mismo sistema sin producir cambios adicionales cuando el estado deseado ya se ha alcanzado. Esta propiedad resulta especialmente útil en entornos que deben reconstruirse o validarse varias veces, ya que reduce el riesgo de que una segunda ejecución altere de forma inesperada una configuración que ya era correcta.

Desde el punto de vista de la seguridad, la gestión de configuración también tiene un papel relevante. Mantener sistemas en un estado conocido permite reducir desviaciones de configuración, aplicar políticas de forma coherente y comprobar que los sistemas no se alejan del estado previsto. Esta idea coincide con los principios de gestión de configuración orientada a la seguridad, donde el objetivo no es únicamente desplegar sistemas, sino mantenerlos bajo configuraciones controladas y verificables, tal como recogen guías y recomendaciones técnicas de gestión segura de la configuración [6, 7].

En consecuencia, la automatización utilizada en este trabajo no se plantea como una simple ejecución de scripts, sino como un flujo estructurado por capas. Terraform se emplea para definir y crear la infraestructura virtual, mientras que PowerShell DSC se utiliza para configurar declarativamente los roles de Windows desplegados sobre dicha infraestructura. Esta separación permite asignar a cada herramienta una responsabilidad concreta y evita mezclar la creación de recursos virtuales con la configuración interna del sistema operativo.

2.2. Terraform como herramienta de aprovisionamiento

Terraform es una herramienta de Infraestructura como Código orientada a definir, crear y modificar recursos de infraestructura mediante ficheros de configuración. Su uso no se limita a entornos cloud públicos (AWS, GCP, Azure), ya que también puede gestionar recursos en plataformas locales o privadas siempre que exista un proveedor compatible. En este trabajo, Terraform se emplea como capa de aprovisionamiento sobre Proxmox, permitiendo definir máquinas virtuales, recursos de cómputo, almace-

namiento y parámetros de red de forma reproducible [11].

El funcionamiento de Terraform se basa en tres elementos principales: los ficheros de configuración, los proveedores y el estado. Los ficheros de configuración describen los recursos que deben existir empleando el lenguaje HCL (*HashiCorp Configuration Language*); los proveedores actúan como intermediarios entre Terraform y la plataforma gestionada; y el fichero de estado (`terraform.tfstate`) permite mantener la relación entre la configuración declarada y los recursos realmente creados [12].

El uso del fichero de estado (`terraform.tfstate`) permite a Terraform disponer de capacidades avanzadas, destacando la construcción de un grafo de recursos interno que calcula dependencias entre elementos, así como la detección de desviaciones de configuración (*Configuration Drift*). Si un recurso virtual sufre una alteración manual (ej: se añaden núcleos, se modifica la RAM) fuera del flujo de código, Terraform puede detectar esa desviación durante la fase de planificación y mostrar las acciones necesarias para reconciliar el estado real de la infraestructura con el estado declarado en los archivos de configuración.

El flujo de trabajo habitual de Terraform se estructura en fases operativas bien diferenciadas que garantizan la integridad del despliegue. Tras escribir la configuración en HCL, el flujo comienza con la inicialización del entorno mediante `terraform init`, que prepara el directorio de trabajo y descarga los componentes o proveedores necesarios. A continuación, se ejecuta (de manera opcional) la fase de validación estática a través de `terraform validate`, un comando crítico que verifica la corrección sintáctica e interna de los ficheros de configuración sin interactuar con la infraestructura real. Una vez asegurada la validez del código, se realiza la planificación mediante `terraform plan`, que permite revisar visualmente qué acciones de aprovisionamiento se van a ejecutar sobre el hipervisor. Finalmente, se aplica el plan mediante `terraform apply`, creando, modificando o eliminando los recursos virtuales necesarios. Esta separación estricta entre definición, validación, planificación y aplicación resulta clave en entornos de producción donde se busca reducir cambios manuales no controlados y aumentar la trazabilidad de la infraestructura.

En el contexto de este TFG, Terraform se utiliza para resolver una parte concreta del problema: crear de forma repetible las máquinas virtuales del laboratorio a partir de una plantilla base. Esta capa incluye la definición de los nodos, la asignación de vCPU, memoria RAM, discos, interfaces de red, direcciones IP y almacenamiento adicional cuando es necesario. Sin embargo, Terraform no se utiliza para configurar en profundidad los servicios internos de Windows. Esa responsabilidad se delega en DSC, que actúa una vez que las máquinas ya existen y están operativas.

Esta separación de responsabilidades constituye una de las decisiones principales del

diseño arquitectónico del proyecto. Una forma sencilla de entender la relación entre ambas herramientas es compararla con la construcción de un edificio: Terraform se encarga de preparar el terreno, levantar la estructura, colocar las paredes y entregar las llaves; DSC entra después para convertir ese edificio vacío en un espacio habitable, instalando los servicios, configurando las estancias, asignando permisos de acceso y comprobando que todo queda en el estado previsto. En términos técnicos, Terraform construye el continente, es decir, la infraestructura virtual base; mientras que DSC configura el contenido, aplicando de forma granular el estado deseado de cada rol del sistema operativo.

Por este motivo, Terraform resulta adecuado para declarar infraestructura virtual, pero no es la herramienta más natural para expresar configuraciones detalladas de Active Directory, permisos NTFS, recursos compartidos SMB o lógica específica del sistema operativo Windows. El trabajo utiliza Terraform para construir la base sobre la que posteriormente DSC aplica el estado deseado de cada nodo.

2.3. Desired State Configuration: arquitectura, funcionamiento y evolución

Desired State Configuration (DSC) es una tecnología de gestión de configuración orientada a definir el estado deseado de un sistema mediante código. En lugar de describir una secuencia imperativa de pasos, DSC permite declarar cómo debe quedar configurado un nodo: qué roles deben estar instalados, qué servicios deben existir, qué archivos deben estar presentes, qué usuarios o grupos deben crearse o qué parámetros del sistema deben cumplirse [13].

En su versión clásica, PowerShell DSC se integra directamente con Windows PowerShell y utiliza configuraciones escritas en PowerShell que posteriormente se compilan en documentos MOF (*Managed Object Format*), basados en el estándar CIM (*Common Information Model*). Estos documentos son aplicados por el *Local Configuration Manager* (LCM), componente encargado de interpretar la configuración, resolver las dependencias declaradas entre recursos y aplicar el estado esperado del sistema mediante los recursos DSC correspondientes [14, 15].

El modelo de recursos de DSC se basa en tres operaciones principales: **Get**, **Test** y **Set**. La operación **Get** obtiene el estado actual del recurso, **Test** comprueba si dicho estado coincide con el declarado en la configuración, y **Set** aplica los cambios necesarios cuando existe una discrepancia. El LCM invoca estas operaciones sobre los recursos definidos en la configuración, lo que permite aproximar el sistema al estado deseado de forma idempotente.

La Figura 2.1 muestra de forma simplificada la arquitectura de funcionamiento de DSC en el modelo clásico utilizado en este trabajo.

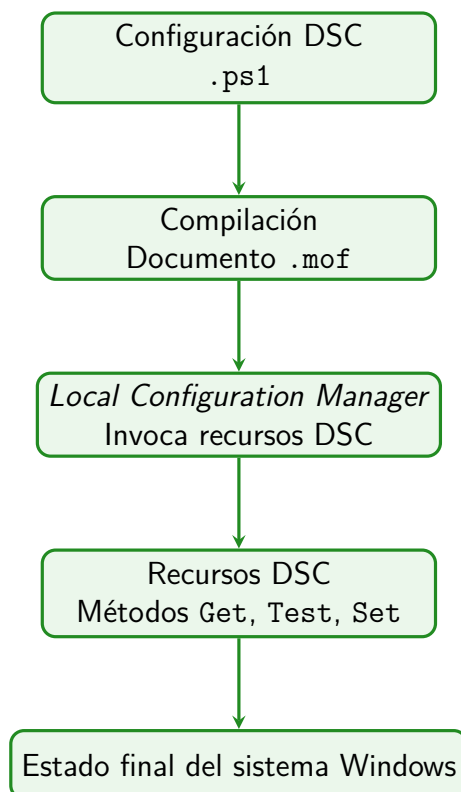


Figura 2.1: Arquitectura simplificada de PowerShell DSC en el modelo clásico.

El funcionamiento de DSC se apoya en recursos. Un recurso DSC encapsula la lógica necesaria para comprobar y aplicar una parte concreta de la configuración del sistema. Por ejemplo, un recurso puede encargarse de instalar una característica de Windows, crear un directorio, configurar un servicio, modificar una clave del registro o ejecutar una acción idempotente. Esta separación permite construir configuraciones más estructuradas y facilita que un mismo recurso pueda reutilizarse en distintos escenarios.

DSC puede aplicarse principalmente mediante dos modelos de arquitectura: el modo *push* y el modo *pull*. En el modo *pull*, los nodos consultan de forma periódica un servidor o servicio centralizado para obtener sus configuraciones. Por el contrario, el presente trabajo utiliza el modelo *push*, ya que buscamos aplicar las configuraciones directamente y no se requiere desplegar una infraestructura adicional de distribución de configuraciones. Figura 2.2 [16].

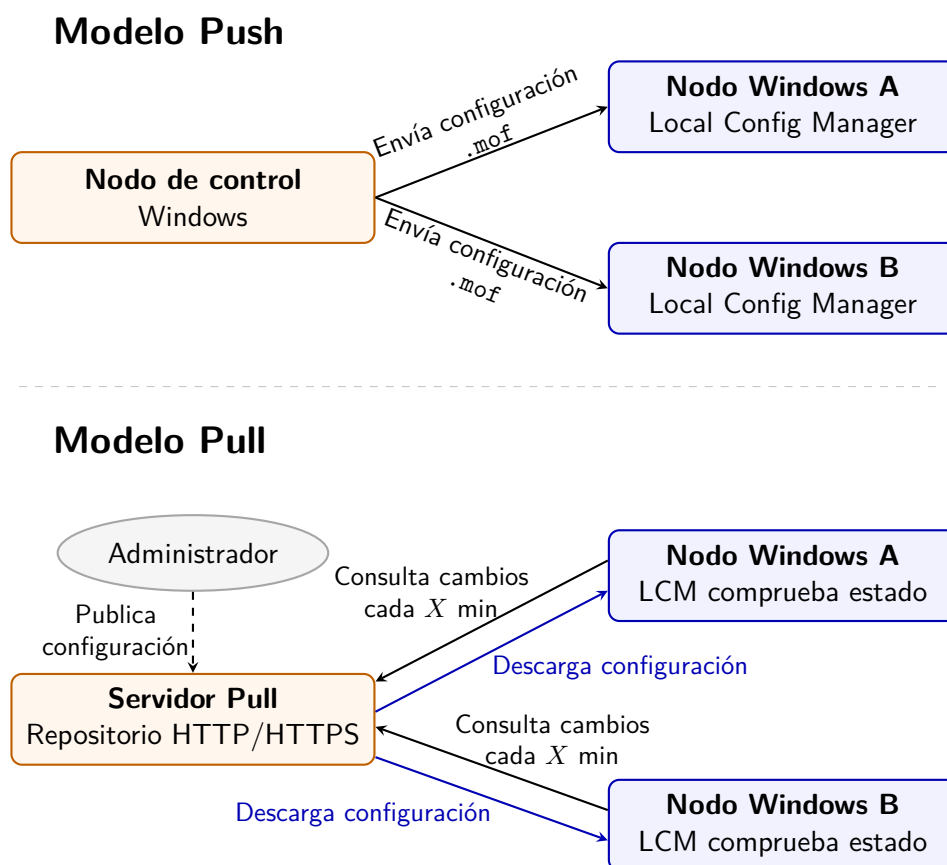


Figura 2.2: Comparación entre los modelos Push y Pull en PowerShell DSC.

En este modelo *push*, el cmdlet `Start-DscConfiguration` es el encargado de aplicar la configuración MOF sobre los nodos de destino. Al ejecutarse, este cmdlet interactúa con el LCM de la máquina objetivo para iniciar la convergencia del sistema. De forma predeterminada, opera mediante tareas en segundo plano, lo que permite controlar la aplicación del estado declarado sin bloquear la sesión administrativa.

La administración técnica de este comando permite, además, dos variantes operativas relevantes para el control de la infraestructura del laboratorio:

- **Uso de sesiones CIM:** La especificación de los equipos remotos a los que se desea aplicar el estado declarado puede realizarse indicando sus nombres de red o mediante el establecimiento explícito de sesiones basadas en CIM.
- **Parámetro `-UseExisting`:** Cuando el cmdlet se invoca junto con el parámetro `-UseExisting`, se indica al LCM que vuelva a aplicar la última configuración almacenada localmente en el equipo de destino. Esto resulta útil para corregir desviaciones técnicas detectadas sin necesidad de retransmitir el archivo MOF desde el nodo de origen.

La evolución de DSC también resulta relevante para justificar las decisiones de la arquitectura del proyecto. El paso desde PowerShell DSC 1.1 hacia las versiones posteriores no representa únicamente una actualización incremental, sino un cambio importante en el modelo de ejecución, en la relación con PowerShell y en el tipo de escenarios para los que resulta más adecuada cada versión. La Tabla 2.1 resume esta evolución de forma comparativa.

Tabla 2.1: Comparativa entre DSC 1.1, DSC 2.0 y Microsoft DSC 3.0

Versión	Base tecnológica	LCM / Pull	Formato de configuración	Enfoque principal	Adecuación al TFG
DSC 1.1	Windows PowerShell 5.1; módulo PSDSC integrado	Sí / Sí	Configuraciones en PowerShell compiladas a MOF	Gestión declarativa clásica de sistemas Windows, con administración remota, recursos DSC tradicionales y mantenimiento del estado deseado	Muy alta: encaja con Windows Server, PowerShell 5.1, WinRM y el modelo clásico de configuración remota
DSC 2.0	Módulo PSDSC 2.x para PowerShell moderno	No / No	Invocación directa de recursos con "Invoke-DscResource"; sin el modelo clásico completo	Uso soportado principalmente con Azure Machine Configuration; enfoque más limitado que DSC 1.1	Baja: Microsoft recomienda DSC 1.1 fuera de Azure Machine Configuration; además desaparecen LCM y cmdlets clásicos
DSC 3.0	Plataforma independiente; herramienta <code>dsc</code> ; compatible con Windows, Linux y macOS	No / No	Documentos declarativos en JSON o YAML	Plataforma moderna y multiplataforma, desacoplada de PowerShell y orientada a un nuevo modelo de configuración	Media-baja en este caso: relevante como evolución futura, pero menos alineada con el flujo clásico Windows/PowerShell del proyecto

PowerShell DSC 1.1 representa el modelo clásico de DSC integrado en Windows PowerShell 5.1. Esta versión se basa en configuraciones escritas en PowerShell que se compilan a documentos MOF y son aplicadas por el *Local Configuration Manager*. Una ventaja arquitectónica clave de este modelo es que el LCM actúa como un motor persistente en segundo plano; evalúa de forma periódica el estado del nodo según un intervalo configurable para verificar que no se hayan producido cambios respecto a la configuración aplicada (*configuration drift*), permitiendo corregir automáticamente cualquier diferencia. Todo ello resulta especialmente adecuado para el presente trabajo porque permite aplicar configuraciones completas sobre nodos Windows mediante cmdlets clásicos como `Start-DscConfiguration`, utilizar sesiones CIM o WinRM y apoyarse en recursos consolidados del ecosistema Windows.

PowerShell DSC 2.0, asociado al ecosistema PowerShell moderno, introdujo cambios

significativos respecto al modelo anterior. Entre ellos destaca la desaparición del LCM, del servidor *pull* y de varios cmdlets clásicos, lo que limita su uso en escenarios que dependen del modelo tradicional de configuración remota. Al prescindir del LCM local, la responsabilidad de la periodicidad y del control continuo de la configuración se traslada a agentes externos. Así, su ámbito de aplicación queda más vinculado a contextos específicos, como Azure Machine Configuration, que al despliegue local de infraestructuras Windows mediante el modelo clásico [17].

Microsoft DSC 3.0 supone una evolución más profunda del modelo. A diferencia de PowerShell DSC, se plantea como una herramienta independiente de PowerShell, con soporte para documentos de configuración en JSON o YAML y una orientación multiplataforma. Esta evolución resulta relevante como línea futura, pero no es la opción más alineada con el objetivo práctico de este TFG, centrado en automatizar roles tradicionales de Windows Server mediante recursos y patrones ya consolidados.

Por tanto, la elección de PowerShell DSC 1.1 no se debe a que sea la versión más reciente, sino a que es la más adecuada para el escenario concreto del proyecto. El laboratorio requiere configurar servicios estrechamente ligados al ecosistema Windows, como Active Directory, DNS, permisos NTFS y recursos compartidos SMB. En este contexto, el modelo clásico basado en MOF, LCM, WinRM y recursos DSC maduros proporciona una base más estable y coherente para el flujo de automatización desarrollado.

2.4. Ansible y automatización de sistemas Windows

Ansible es una herramienta de automatización ampliamente utilizada para la gestión de configuración, el despliegue de aplicaciones y la orquestación de tareas en infraestructura. A diferencia de Terraform, cuyo propósito principal es el aprovisionamiento de la infraestructura virtual, Ansible se orienta a ejecutar tareas sobre sistemas operativos ya existentes para aproximarlos a un estado determinado mediante el uso de *playbooks*.

Una de las particularidades de Ansible es su arquitectura sin agente (*agentless*). Al contrario que otras soluciones, no requiere la instalación de un servicio permanente propio de Ansible en los nodos de destino; en su lugar, un nodo de control se conecta de forma remota a las máquinas objetivo, ejecuta los módulos necesarios y cierra la sesión. En entornos Windows, esta comunicación se realiza habitualmente mediante WinRM, PSRP u OpenSSH, lo que exige preparar previamente los mecanismos de administración remota del sistema [18, 19].

Bajo este modelo, Ansible puede apoyarse directamente en DSC mediante el mó-

dulo especializado `win_dsc`. Esta integración permite invocar recursos DSC desde los propios *playbooks*, lo que demuestra que ambas tecnologías no son excluyentes. En infraestructuras híbridas o de gran volumen, Ansible puede actuar como orquestador general del flujo de trabajo, delegando en recursos DSC la lógica específica necesaria para configurar componentes propios del ecosistema Windows [20].

A pesar de la versatilidad de este enfoque, se ha decidido no incorporar Ansible como tecnología principal de la arquitectura por criterios de diseño y homogeneidad tecnológica. El objetivo del TFG es estudiar la automatización del despliegue y configuración de un entorno Windows mediante tecnologías estrechamente vinculadas al propio ecosistema Microsoft. En este contexto, introducir Ansible implicaría añadir una capa de orquestación externa y operar desde un nodo de control adicional, normalmente basado en Linux o en otro entorno compatible, ya que Windows no se utiliza como nodo de control nativo de Ansible [18].

En segundo lugar, la diferencia en el ciclo de vida de la configuración también favorece el uso directo de DSC en este proyecto. Ansible opera bajo un modelo de ejecución bajo demanda: el estado se evalúa cuando se lanza el *playbook* desde el nodo de control. En cambio, el modelo clásico de DSC 1.1 se apoya en el *Local Configuration Manager* (LCM), un componente local del sistema Windows que aplica configuraciones y puede configurarse para monitorizar o corregir desviaciones respecto al estado deseado mediante modos como `ApplyAndMonitor` o `ApplyAndAutoCorrect` [14]. Esta característica resulta especialmente alineada con un laboratorio en el que se busca mantener coherencia entre la configuración declarada y el estado real de los nodos.

Por tanto, la combinación de Terraform y PowerShell DSC 1.1 proporciona una solución más autocontenida y coherente con el alcance de este laboratorio. Terraform resuelve el ciclo de vida del aprovisionamiento de la infraestructura virtual sobre Proxmox, mientras que DSC asume la configuración granular del sistema operativo aprovechando su integración con PowerShell, el LCM y los recursos clásicos del ecosistema Windows. Ansible se considera una alternativa válida y complementaria, especialmente en entornos heterogéneos o de mayor escala, pero no la herramienta principal de la solución adoptada.

2.5. Comparativa tecnológica y justificación de la solución adoptada

La elección tecnológica del trabajo se fundamenta en separar claramente dos dominios del ciclo de vida del despliegue: el aprovisionamiento de la infraestructura virtual y la configuración interna del sistema operativo. Para el primer dominio se adopta Terraform, mientras que para el segundo se adopta PowerShell DSC. Esta división evi-

ta forzar a una única herramienta a asumir responsabilidades con enfoques diferentes y permite aprovechar las fortalezas de cada solución en la fase para la que está diseñada.

El análisis no se limita únicamente a las tecnologías principales del proyecto. Además de Terraform y PowerShell DSC, se han considerado alternativas de gestión de configuración y orquestación como Ansible, así como soluciones más orientadas a entornos empresariales de gran escala como Puppet, Chef o Salt. También se incluye Cloudbase-Init, no como herramienta principal de configuración, sino como componente auxiliar en la inicialización de instancias Windows dentro del entorno virtualizado.

La Tabla 2.2 resume la comparación entre las tecnologías consideradas y el papel que desempeñan dentro del contexto del TFG.

Tabla 2.2: Comparativa de tecnologías de automatización consideradas

Tecnología	Ámbito principal	Ventajas relevantes	Uso en el TFG
Terraform	Aprovisionamiento de infraestructura	Definición declarativa de recursos, planificación de cambios, proveedores, grafo de dependencias y gestión de estado.	Principal: creación de máquinas virtuales en Proxmox, red, discos y parámetros iniciales.
PowerShell DSC 1.1	Gestión de configuración Windows	Integración con PowerShell, LCM, recursos específicos para Windows, modelo declarativo e idempotente.	Principal: configuración de controlador de dominio, servidor de ficheros, cliente, permisos y servicios.
Ansible	Gestión de configuración y orquestación	Arquitectura sin agente, playbooks en YAML, amplio ecosistema e integración con Windows mediante WinRM, SSH o DSC.	Alternativa/complemento: analizado como opción válida, pero no adoptado como herramienta principal.
Puppet/Chef/Salt	Gestión de configuración a escala	Soluciones maduras para entornos amplios, heterogéneos y con gestión centralizada.	Fuera del alcance práctico: complejidad adicional y menor alineación con el foco Windows/PowerShell del proyecto.
Cloudbase-Init	Inicialización de instancias Windows	Parametrización inicial de clones Windows, integración con metadatos y apoyo al primer arranque.	Complementaria: apoyo en la fase de inicialización, no como herramienta principal de configuración.

La solución adoptada no busca demostrar que Terraform o DSC sean superiores en términos absolutos al resto de alternativas, sino justificar su adecuación al escenario concreto del proyecto. Terraform encaja con la necesidad de crear infraestructura virtual de forma repetible sobre Proxmox, mientras que DSC encaja con la necesidad de configurar estados internos de sistemas Windows de forma **nativa**. La combinación de ambas herramientas permite construir un flujo por capas: primero se crea la infraestructura, después se configura el sistema operativo y finalmente se valida el resultado.

La elección de DSC frente a Ansible se justifica especialmente por el tipo de configuración abordada. El trabajo requiere automatizar roles y componentes propios de Windows, como Active Directory, DNS, servidor de ficheros, permisos NTFS, recursos SMB, cuentas administrativas, grupos y configuración local. **DSC** proporciona un modelo **nativo** para expresar este tipo de configuraciones mediante recursos y PowerShell, reduciendo la distancia entre la herramienta empleada y el sistema gestionado.

Ansible presenta ventajas claras en escenarios donde se requiere orquestación desde un nodo de control externo, administración de sistemas heterogéneos o integración con un ecosistema más amplio de automatización. Además, su capacidad para invocar recursos DSC mediante el módulo `win_dsc` demuestra que ambas tecnologías no son incompatibles, sino potencialmente complementarias. Sin embargo, esas ventajas no constituyen el eje principal de este TFG, cuyo objetivo es estudiar la automatización de un entorno Windows apoyándose en tecnologías estrechamente vinculadas al propio ecosistema Microsoft.

Por este motivo, el valor técnico de la solución adoptada reside en combinar Terraform y PowerShell DSC de forma especializada. Terraform construye la infraestructura virtual base sobre la que se ejecutan los nodos, mientras que DSC configura el contenido funcional de esos sistemas: roles, servicios, cuentas, grupos, permisos y recursos compartidos. Esta separación de responsabilidades permite mantener una arquitectura clara, reproducible y coherente con el alcance del laboratorio desarrollado.

2.6. Resumen del enfoque seleccionado

A partir del análisis tecnológico realizado, la solución adoptada en este TFG se construye sobre una arquitectura de automatización dividida claramente por responsabilidades. El flujo operativo comienza con la preparación de una imagen base y una plantilla homogénea en el hipervisor; a continuación, Terraform se encarga de aprovisionar las máquinas virtuales y la infraestructura de red necesaria; posteriormente, PowerShell DSC transforma esos nodos recién desplegados en sistemas Windows con roles funcionales; y, finalmente, las pruebas de validación comprueban que el entorno resultante cumple los requisitos definidos.

Esta combinación de herramientas responde de manera directa a los desafíos planteados en el Capítulo 1. El despliegue manual y propenso a errores se sustituye por una definición reproducible de la infraestructura; la configuración interactiva de roles del sistema operativo se reemplaza por configuraciones declarativas controladas; y el bastionado de seguridad se integra dentro del flujo de validación para comprobar que las directivas aplicadas no comprometen la continuidad operativa ni la capacidad de

administración del entorno.

Por tanto, el estudio del estado del arte no conduce a la adopción de una única herramienta aislada, sino al diseño de una solución compuesta y complementaria. Terraform aporta consistencia en la capa de infraestructura virtual, DSC proporciona control declarativo en la capa de configuración de Windows Server, y las guías CCN-STIC actúan como marco técnico de referencia para el endurecimiento del sistema. Esta separación de dominios permite justificar el criterio técnico de la solución implementada y evita que el proyecto se limite a una colección de scripts inconexos, consolidando una arquitectura clara, reproducible y alineada con los objetivos del trabajo.

3. Metodología y Requisitos del Sistema

3.1. Metodología de desarrollo

La metodología seguida en este trabajo se basa en un desarrollo progresivo por fases. Cada etapa prepara las condiciones necesarias para la siguiente, de forma que el proceso completo pueda entenderse como una cadena de automatización que comienza con la preparación del medio de instalación y finaliza con la validación del entorno tras el bastionado.

El flujo metodológico general puede observarse en la Figura 3.1.

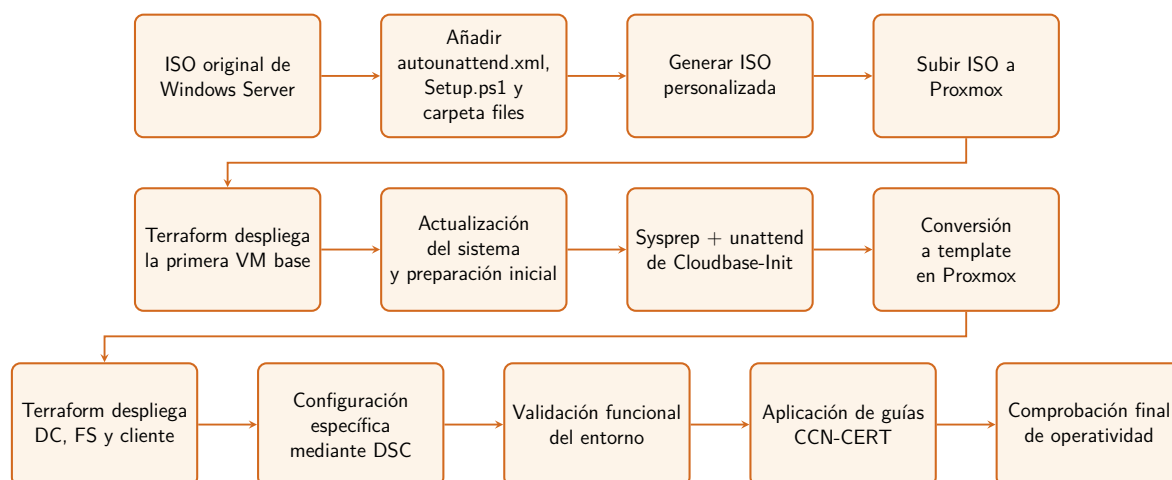


Figura 3.1: Flujo de generación de la plantilla y despliegue automatizado del entorno.

La primera fase (vinculada con el OE-01) consiste en preparar una imagen personalizada de Windows Server. Para ello se parte de una ISO original a la que se incorporan el archivo `autounattend.xml`, el script `Setup.ps1` y los componentes auxiliares necesarios para la instalación desatendida, la virtualización y la configuración posterior.

La segunda fase (vinculada también con el OE-01) consiste en desplegar una primera máquina base en Proxmox a partir de la ISO personalizada. Esta máquina se actuali-

za, se prepara para administración remota y se deja lista para su generalización. Una vez completada esta preparación, se ejecuta Sysprep y la máquina se convierte en una plantilla reutilizable.

La tercera fase (vinculada con el OE-02) corresponde al aprovisionamiento automatizado de los nodos finales mediante *Terraform*. A partir de la plantilla generada en la fase anterior, se crean automáticamente las máquinas virtuales destinadas a controlador de dominio, servidor de ficheros y cliente Windows, asignando a cada una los recursos de hardware y red necesarios.

La cuarta fase (vinculada con el OE-03) consiste en aplicar las configuraciones declarativas mediante *PowerShell DSC*. En esta etapa, cada nodo recibe la configuración correspondiente a su rol: el controlador de dominio levanta el Directorio Activo y el DNS; el servidor de ficheros se integra en el dominio para configurar el almacenamiento y los recursos compartidos; y el cliente Windows se une al entorno para poder interactuar con los servicios.

La quinta fase (vinculada con el OE-04) se centra en la preparación de la administración y la continuidad del entorno. Antes de aplicar el bastionado, se definen las cuentas alternativas de tipo *BreakGlass* y se configuran sus permisos en los grupos locales y de dominio, asegurando que no se perderá el acceso de gestión tras aplicar las políticas de seguridad restrictivas.

La sexta fase (vinculada con el OE-05) corresponde a la validación funcional inicial del entorno. En ella se comprueba de forma experimental que el dominio está operativo, que los nodos se integran bien, que el servidor de ficheros comparte los recursos con los permisos NTFS correctos y que el cliente responde bien a la autenticación antes de alterar la seguridad.

Finalmente, la séptima fase (vinculada con el OE-04 y el OE-05) incorpora la aplicación de las medidas de bastionado basadas en las guías del CCN-CERT y la comprobación final de todo el entorno. Esta última validación sirve para evaluar si la infraestructura conserva su funcionalidad esencial y su capacidad de administración tras haber sido endurecida.

3.2. Alcance y limitaciones

El alcance de este TFG se centra en el diseño, implementación y validación de un flujo automatizado para desplegar y configurar un entorno Windows de laboratorio con varios roles diferenciados. La solución desarrollada no pretende cubrir todos los escenarios posibles de una infraestructura empresarial real, sino construir un entorno repre-

sentativo que permita evaluar el uso combinado de instalación desatendida, plantillas reutilizables, Terraform, PowerShell DSC y bastionado basado en las guías CCN-STIC.

En concreto, la solución implementada cubre el despliegue de tres nodos principales: un controlador de dominio, un servidor de ficheros y un cliente Windows. Estos nodos permiten reproducir una infraestructura mínima pero funcional, en la que existen servicios de identidad, resolución de nombres, recursos compartidos, autenticación de usuarios, permisos basados en grupos y validación desde un equipo final unido al dominio.

La Tabla 3.1 resume el alcance funcional cubierto por la solución.

Tabla 3.1: Alcance funcional de la solución implementada

Elemento	Alcance cubierto
Imagen base	Preparación de una instalación desatendida de Windows Server, incorporación de componentes necesarios (como Cloudbase-Init) y generación de una base reutilizable.
Plantilla virtual	Creación de una plantilla en Proxmox a partir de la máquina base preparada, permitiendo desplegar nuevos nodos con una configuración inicial homogénea.
Aprovisionamiento	Despliegue automatizado de las máquinas virtuales necesarias mediante Terraform, incluyendo recursos de cómputo, red y almacenamiento.
Controlador de dominio	Instalación de los roles necesarios, promoción del servidor, configuración DNS y creación de una estructura básica de Active Directory.
Active Directory	Creación de unidades organizativas, usuarios y grupos necesarios para las pruebas funcionales y para el modelo de permisos del servidor de ficheros.
Servidor de ficheros	Unión al dominio, preparación del almacenamiento, creación de recursos compartidos SMB, configuración de permisos NTFS y SMB, y activación de mecanismos como <i>Access-Based Enumeration</i> .
Cliente Windows	Integración en el dominio y utilización como nodo de validación para comprobar autenticación, resolución DNS, acceso a recursos compartidos y aplicación efectiva de permisos.
Bastionado	Aplicación de configuraciones basadas en guías CCN-STIC sobre los nodos del laboratorio y comprobación de que el entorno mantiene su operatividad.
Continuidad administrativa	Creación de cuentas <i>BreakGlass</i> y asignación controlada de permisos para conservar la capacidad de gestión tras aplicar políticas de seguridad.
Validación	Ejecución de pruebas funcionales antes y después del bastionado, centradas en dominio, DNS, unión de nodos, acceso a recursos, permisos y administración.

Desde el punto de vista técnico, el trabajo cubre el ciclo completo de construcción del laboratorio: preparación de la imagen base, creación de la plantilla, aprovisionamiento de nodos, configuración declarativa por rol, aplicación de medidas de seguridad y validación posterior. Por tanto, el alcance no se limita a demostrar que una máquina

virtual puede crearse automáticamente, sino que incluye la configuración funcional de servicios reales y la comprobación de su comportamiento desde un cliente del dominio.

Sin embargo, el trabajo queda acotado a un entorno de laboratorio controlado. La infraestructura resultante reproduce una arquitectura mínima suficiente para validar el flujo de automatización, pero no pretende sustituir el diseño completo de una red corporativa en producción. Esta decisión permite concentrar el desarrollo en la reproducibilidad, la configuración declarativa y la validación técnica, evitando dispersar el trabajo en elementos que excederían el objetivo principal del TFG.

La Tabla 3.2 recoge las principales limitaciones asumidas durante el desarrollo.

Tabla 3.2: Limitaciones del trabajo

Ámbito	Limitación asumida
Escala del entorno	El laboratorio se limita a tres nodos principales: controlador de dominio, servidor de ficheros y cliente Windows. No se evalúa un despliegue masivo con decenas o cientos de máquinas.
Alta disponibilidad	No se implementan controladores de dominio redundantes, replicación entre sedes, clústeres de ficheros ni mecanismos avanzados de tolerancia a fallos.
Producción empresarial	El entorno se diseña para validación académica y técnica en laboratorio, no como una arquitectura directamente desplegable en producción sin adaptaciones adicionales.
Seguridad integral	El bastionado se basa en guías CCN-STIC aplicadas al entorno, pero no se realiza una auditoría completa de cumplimiento del ENS ni una certificación formal de seguridad.
Gestión de identidades	Se crea una estructura básica de Active Directory con usuarios, grupos y unidades organizativas suficientes para las pruebas, pero no se aborda un modelo corporativo completo de identidad, delegación y ciclo de vida de cuentas.
Gestión de secretos	Las credenciales utilizadas durante la automatización se gestionan de forma adecuada para el laboratorio, pero no se implementa una solución empresarial de custodia, rotación y auditoría de secretos.
Monitorización y respuesta	No se incluye una plataforma completa de monitorización, SIEM, EDR ni respuesta ante incidentes. La validación se centra en pruebas funcionales y operativas del entorno desplegado.
Servicios adicionales	Quedan fuera del alcance servicios como PKI corporativa, WSUS, backup empresarial, despliegue de software, inventario centralizado o gestión avanzada de parches.
Evaluación ofensiva	No se realiza una prueba de intrusión completa ni una evaluación ofensiva formal. Las comprobaciones de seguridad se orientan a verificar la aplicación del bastionado y la continuidad administrativa.

Estas limitaciones no invalidan el trabajo, sino que delimitan su propósito. El ob-

jetivo no es construir una infraestructura corporativa completa, sino demostrar que es posible automatizar de forma reproducible un entorno Windows funcional, aplicar configuraciones de seguridad sobre él y comprobar que los servicios esenciales siguen operativos. De este modo, el alcance queda alineado con los objetivos definidos en el Capítulo 1: preparar una base reutilizable, aprovisionar infraestructura, configurar roles mediante DSC, aplicar bastionado y validar empíricamente el resultado.

3.3. Requisitos del sistema y del entorno

Para alcanzar los objetivos definidos, el entorno tiene que cumplir una serie de condiciones mínimas. No basta con que Terraform cree las máquinas virtuales, sino que estas deben poder configurarse después, integrarse en el dominio y quedar en un estado comprobable. Por ello, se definen requisitos relacionados con el despliegue, la configuración, la validación y la administración posterior del sistema. Estos requisitos se agrupan en funcionales y no funcionales.

Los requisitos funcionales hacen referencia a lo que debe ser capaz de hacer la solución desarrollada. En este TFG se centran principalmente en generar una imagen base de Windows, crear las máquinas virtuales en Proxmox mediante Terraform, configurar los roles principales con DSC y comprobar que los nodos desplegados funcionan correctamente dentro del laboratorio.

Tabla 3.3: Requisitos funcionales del entorno

Código	Descripción
RF-01	Permitir la instalación desatendida de una máquina base Windows Server a partir de una imagen personalizada.
RF-02	Generar una plantilla reutilizable en Proxmox preparada para el despliegue de nuevos nodos.
RF-03	Aprovisionar mediante Terraform las máquinas virtuales que componen el laboratorio.
RF-04	Configurar mediante DSC un controlador de dominio con los servicios y estructura básica necesarios.
RF-05	Configurar mediante DSC un servidor de ficheros integrado en el dominio, con recursos compartidos y permisos basados en grupos.
RF-06	Configurar un cliente Windows integrado en el dominio para validar el uso de los servicios desplegados.
RF-07	Definir cuentas administrativas alternativas y permisos adecuados para mantener la administración tras el bastionado.
RF-08	Aplicar medidas de bastionado basadas en guías del CCN-CERT y comprobar su impacto sobre el entorno.

Además, también se han definido requisitos no funcionales. En este caso no se describe una acción concreta del sistema, sino características que afectan a la forma en la que se realiza la automatización. Entre ellas se incluyen que el despliegue pueda repetirse, que los scripts estén separados por fases, que sea posible mantenerlos sin rehacer todo el proceso y que las dependencias externas estén controladas.

Tabla 3.4: Requisitos no funcionales del entorno

Código	Descripción
RNF-01	El despliegue debe ser reproducible, de forma que el laboratorio pueda reconstruirse con un comportamiento equivalente entre ejecuciones.
RNF-02	La solución debe ser modular, separando la preparación de imagen, el aprovisionamiento de infraestructura y la configuración del sistema operativo.
RNF-03	Las configuraciones deben ser mantenibles y estar expresadas mediante artefactos versionables.
RNF-04	El entorno debe reducir la dependencia de descargas externas durante fases críticas, incorporando previamente componentes y módulos necesarios.
RNF-05	La solución debe permitir validar tanto el estado funcional inicial como el comportamiento posterior al bastionado.
RNF-06	El proceso debe adaptarse a un entorno de laboratorio con recursos físicos limitados.

Esta separación ayuda a relacionar los objetivos del trabajo con las fases que se explican más adelante. También permite justificar mejor las decisiones tomadas durante la implementación, ya que cada parte del proceso responde a una necesidad concreta del entorno y no queda presentada únicamente como una lista de comandos o configuraciones.

3.4. Entorno de trabajo y herramientas empleadas

El proyecto se desarrolla en un entorno de laboratorio virtualizado, diseñado para reproducir de forma controlada un escenario básico de administración de sistemas Windows. La plataforma de virtualización empleada es Proxmox, sobre la que se despliegan las máquinas virtuales que componen el laboratorio. Esta elección permite trabajar con plantillas reutilizables, gestionar recursos de forma flexible y repetir el despliegue sin depender de varios equipos físicos independientes.

El entorno físico utilizado parte de un portátil reutilizado y adaptado al que se le cambió el procesador y la memoria RAM para soportar la infraestructura de virtualización. Aunque este aspecto no constituye el núcleo del trabajo, condiciona el diseño del laboratorio y refuerza la necesidad de emplear plantillas, automatización y reutilización de recursos para construir un escenario funcional con medios limitados.

La solución combina distintas herramientas, cada una de ellas asociada a una fase concreta del flujo de trabajo. La Tabla 3.5 resume el papel principal de cada tecnología dentro del proyecto.

Tabla 3.5: Herramientas empleadas en el proyecto

Herramienta	Uso dentro del TFG
Proxmox VE	Plataforma de virtualización utilizada para crear, ejecutar y gestionar las máquinas virtuales del laboratorio.
Windows Server	Sistema base empleado para la imagen personalizada, el controlador de dominio y el servidor de ficheros.
Cliente Windows	Sistema utilizado como equipo final para validar la integración en el dominio y el acceso a recursos.
Terraform	Herramienta de aprovisionamiento utilizada para crear la máquina base y los nodos finales en Proxmox.
Cloudbase-Init	Componente utilizado para facilitar la parametrización inicial de máquinas Windows clonadas desde la plantilla.
Sysprep	Herramienta empleada para generalizar la máquina base antes de convertirla en plantilla reutilizable.
PowerShell DSC	Tecnología utilizada para definir y aplicar la configuración declarativa de los nodos Windows.
CCN-CERT	Fuente de guías de bastionado utilizadas para reforzar la seguridad del entorno y comprobar su impacto.

Como se ha indicado anteriormente, el laboratorio se compone de tres nodos principales desplegados a partir de la plantilla generada. Cada uno cumple una función concreta dentro del escenario y permite validar una parte diferente de la solución.

Tabla 3.6: Nodos principales del laboratorio

Nodo	Rol	Dirección IP	Función principal
DC1	Controlador de dominio	192.168.137.10	Servicios de dominio, DNS, estructura organizativa, usuarios y grupos.
FS2	Servidor de ficheros	192.168.137.11	Almacenamiento, recursos compartidos y permisos basados en grupos.
CLI	Cliente Windows	192.168.137.12	Validación de integración en dominio y acceso a servicios.

La Figura 3.2 representa de forma esquemática los principales elementos del entorno de trabajo y su relación dentro del flujo de despliegue automatizado.

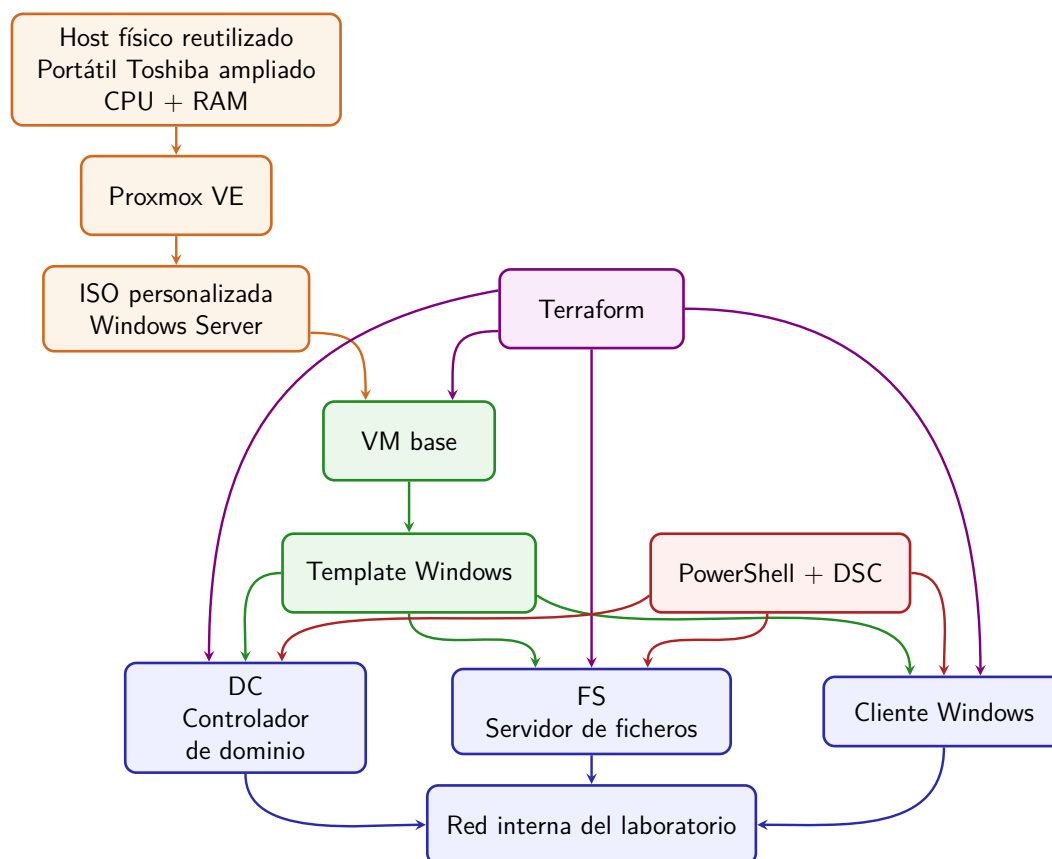


Figura 3.2: Entorno de laboratorio virtualizado y flujo de despliegue.

3.5. Criterios de validación

La validación del trabajo no se limita a comprobar que las herramientas empleadas ejecutan sus scripts o configuraciones sin errores. El objetivo es verificar que el entorno resultante cumple los requisitos definidos y que puede utilizarse de forma efectiva como laboratorio Windows automatizado.

Para ello se establecen una serie de criterios de validación, recogidos en la Tabla 3.7, asociados a las distintas fases del proyecto. Estos criterios permiten comprobar tanto el proceso de despliegue como el estado funcional final de los nodos.

Además de estos criterios generales, en la Tabla 3.8, se definen pruebas concretas que permiten obtener evidencias sobre el estado del entorno. Algunas de estas pruebas se ejecutan antes del bastionado y otras se repiten después, con el fin de comprobar si las medidas de seguridad introducen cambios relevantes en la operatividad del laboratorio.

Tabla 3.7: Criterios de validación del proyecto

Código	Criterio	Comprobación esperada	Fase asociada
CV-01	Imagen base funcional	La instalación desatendida finaliza correctamente y el sistema contiene los componentes previstos.	Preparación de imagen
CV-02	Plantilla reutilizable	La máquina base puede generalizarse y convertirse en plantilla sin impedir el despliegue posterior.	Creación de plantilla
CV-03	Aprovisionamiento reproducible	Terraform crea los nodos definidos con los parámetros esperados.	Aprovisionamiento
CV-04	Configuración del dominio	El controlador de dominio queda operativo y proporciona servicios de dominio y DNS.	DSC - DC
CV-05	Integración de nodos	El servidor de ficheros y el cliente se incorporan correctamente al dominio.	DSC - FS/Cliente
CV-06	Recursos compartidos	El servidor de ficheros ofrece los recursos definidos y aplica permisos basados en grupos.	DSC - FS
CV-07	Validación desde cliente	El cliente puede resolver el dominio, autenticarse y acceder a los recursos permitidos.	Validación funcional
CV-08	Continuidad administrativa	Las cuentas administrativas previstas permiten mantener la gestión del entorno.	Pre-bastionado
CV-09	Operatividad tras bastionado	Los servicios esenciales continúan funcionando tras aplicar las guías CCN-STIC.	Post-bastionado

Tabla 3.8: Pruebas funcionales previstas

Código Prueba	Evidencia esperada	Nodo
PF-01 Comprobar resolución DNS del dominio	El dominio resuelve hacia el controlador de dominio.	Cliente / FS
PF-02 Comprobar estado del dominio	El controlador de dominio responde y contiene la estructura prevista.	DC1
PF-03 Verificar unión al dominio	Los equipos aparecen registrados en Active Directory.	DC1 / FS / Cliente
PF-04 Comprobar estado de DSC	La última configuración aplicada finaliza correctamente.	Todos
PF-05 Validar acceso a recurso compartido	Un usuario autorizado accede al recurso correspondiente.	Cliente / FS
PF-06 Validar denegación de acceso	Un usuario no autorizado no accede a recursos restringidos.	Cliente / FS
PF-07 Comprobar administración remota	El nodo puede administrarse mediante el mecanismo previsto.	DC1 / FS
PF-08 Validar cuenta BreakGlass	La cuenta alternativa conserva acceso administrativo según lo definido.	DC1 / FS / Cliente
PF-09 Comprobar funcionamiento tras CCN-STIC	Los servicios esenciales siguen disponibles tras el bastionado.	Todos

Los criterios y pruebas definidos en este apartado actúan como referencia para la validación posterior. En el Capítulo 4, estas comprobaciones se retoman desde una perspectiva práctica, indicando las evidencias obtenidas y el resultado de cada prueba sobre el entorno desplegado.

4. Diseño, implementación, validación y análisis de resultados

Este capítulo explica cómo se ha construido y probado la solución desarrollada. Para que el desarrollo no quede presentado como una sucesión de herramientas y scripts, el capítulo se divide en cuatro partes: diseño, implementación, validación y análisis de resultados.

Primero se describen las decisiones de diseño que organizan el flujo de trabajo: imagen base, plantilla reutilizable, aprovisionamiento, configuración declarativa, validación y bastionado. Después se explica cómo se implementa cada fase en el laboratorio. A continuación, se presenta el escenario de validación funcional antes y después de aplicar las guías CCN-STIC. Finalmente, se analizan los resultados obtenidos y las incidencias técnicas que condicionaron la solución final.

Los listados completos de código y comandos se recogen en el Anexo A y en el repositorio público asociado al proyecto, accesible mediante el siguiente enlace:

<https://github.com/felipehdzc/tfg-dsc-windows-automation>.

En este capítulo se mantiene la explicación de las decisiones técnicas, las dependencias entre fases y las evidencias obtenidas.

4.1. Diseño de la solución

El diseño de la solución parte de una restricción principal: el entorno debe poder reconstruirse de forma repetible sin depender de una secuencia manual de instalación, configuración y comprobación. Para ello, el flujo se divide en capas con responsabilidades diferenciadas. Cada capa recibe un sistema en un estado concreto, realiza una transformación limitada y entrega a la siguiente fase una base más preparada.

Esta separación evita que una única herramienta asuma todo el proceso. Terraform se emplea para crear infraestructura virtual, Cloudbase-Init para aplicar la parametrización inicial de los clones y PowerShell DSC para configurar el estado interno de Windows. El bastionado se sitúa al final del flujo, pero se tiene en cuenta desde el diseño mediante cuentas BreakGlass y comprobaciones de administración remota.

4.1.1. Arquitectura general y separación por capas

La solución se organiza como una arquitectura por capas. Cada capa resuelve una parte concreta del proceso y entrega a la siguiente un sistema en un estado más preparado. Esta decisión facilita la depuración, evita mezclar responsabilidades y permite comprobar el resultado de cada fase antes de continuar con la siguiente.

El proceso completo comienza con la preparación de una ISO personalizada de Windows Server. Esta ISO incorpora un archivo de respuesta para la instalación desatendida, un script de inicialización y los componentes necesarios para las fases posteriores. A partir de esta ISO se despliega una primera máquina virtual base en Proxmox. Esta VM inicial también se crea mediante Terraform, aunque en esta fase Terraform solo se utiliza para levantar la infraestructura de la máquina y arrancarla desde la ISO personalizada. La configuración interna de Windows queda delegada en la instalación desatendida y en el script `Setup.ps1`.

Una vez preparada la máquina base, se actualiza, se generaliza mediante Sysprep y se convierte en una plantilla reutilizable. A partir de esa plantilla, Terraform se encarga de aprovisionar los nodos finales del laboratorio: el controlador de dominio, el servidor de ficheros y el cliente Windows. Terraform define los recursos virtuales, los parámetros de red y los discos necesarios, mientras que Cloudbase-Init aplica dentro de Windows la parametrización inicial recibida durante el primer arranque de cada clon.

La configuración interna de los sistemas se realiza posteriormente mediante PowerShell DSC. En esta fase, las máquinas desplegadas desde la plantilla dejan de ser instalaciones genéricas de Windows Server y pasan a desempeñar un rol concreto dentro del laboratorio. El controlador de dominio proporciona Active Directory y DNS; el servidor de ficheros se integra en el dominio, prepara el almacenamiento y publica recursos SMB; y el cliente Windows permite validar el comportamiento del entorno desde la perspectiva de un equipo final unido al dominio.

Finalmente, se aplica una fase de validación y bastionado. Primero se comprueba que el entorno funciona correctamente antes de introducir políticas de seguridad. Después se aplican guías CCN-STIC mediante GPO importadas en el dominio y se repiten las comprobaciones esenciales para verificar que la infraestructura sigue siendo funcional y administrable.

La Tabla 4.1 resume las capas principales de la arquitectura.

Tabla 4.1: Capas principales de la solución propuesta

Capa	Herramientas principales	Función dentro del proyecto
Preparación del medio	ISO original, autounattend.xml, Setup.ps1	Instalar Windows Server de forma desatendida y preparar una base común para el resto del flujo.
VM base inicial	Terraform, Proxmox, ISO personalizada	Crear la primera máquina virtual desde la ISO personalizada para preparar la futura plantilla.
Plantilla reutilizable	Proxmox, Sysprep, Cloudbase-Init	Generalizar la máquina base y convertirla en una plantilla desde la que desplegar nodos homogéneos.
Aprovisionamiento	Terraform, Proxmox, configdrive2	Crear los nodos finales del laboratorio, asignando recursos, red, DNS y discos según el rol.
Parametrización inicial	Cloudbase-Init	Aplicar dentro de Windows los metadatos entregados por Terraform durante el primer arranque.
Configuración por rol	PowerShell DSC	Transformar los nodos en controlador de dominio, servidor de ficheros o cliente unido al dominio.
Validación y bastionado	Pruebas funcionales, cuentas BreakGlass, guías CCN-STIC	Comprobar la funcionalidad antes y después del endurecimiento, manteniendo la capacidad de administración.

4.1.2. Decisiones principales de diseño

Además de dividir el flujo en capas, se tomaron varias decisiones de diseño orientadas a mejorar la repetibilidad del despliegue y a reducir dependencias manuales. Estas decisiones no afectan únicamente a la implementación, sino también a la forma en la que el entorno puede validarse y mantenerse.

La Tabla 4.2 resume las decisiones principales.

Tabla 4.2: Decisiones principales de diseño de la solución

Decisión	Motivo	Efecto en la solución
Usar una única plantilla base	Evitar mantener plantillas distintas por rol.	Todos los nodos parten de una base homogénea y se diferencian después mediante configuración.
Crear también la VM base con Terraform	Evitar que la preparación de la plantilla dependa de una creación manual en Proxmox.	El flujo automatizado comienza desde la primera máquina base, no solo desde los clones finales.

Continúa en la siguiente página →

Decisión	Motivo	Efecto en la solución
Separar Terraform y DSC	Distinguir infraestructura virtual y configuración interna del sistema operativo.	Terraform crea máquinas; DSC configura roles, servicios, usuarios, grupos y permisos.
Incluir Cloudbase-Init en la plantilla	Permitir que cada clon reciba parámetros propios en el primer arranque.	La plantilla sigue siendo genérica, pero cada nodo obtiene IP, DNS e identidad específica.
Dividir DSC por fases	Gestionar dependencias entre dominio, DNS, reinicios y permisos.	Se reduce el riesgo de aplicar configuraciones antes de que sus requisitos estén disponibles.
Preparar cuentas BreakGlass antes del bastionado	Evitar pérdida de acceso tras aplicar políticas restrictivas.	La administración se conserva antes y después del endurecimiento.
Validar antes y después de CCN-STIC	Diferenciar errores de despliegue de efectos del bastionado.	Permite comprobar si las políticas de seguridad afectan a servicios esenciales.

Estas decisiones permiten que el flujo no dependa de una ejecución monolítica. Cada fase puede comprobarse de forma independiente y, en caso de error, el punto de fallo queda más acotado.

4.1.3. Diseño de la imagen base y de la plantilla reutilizable

El diseño de la imagen base busca evitar que cada nodo del laboratorio se instale desde cero. Para ello se parte de una ISO personalizada que automatiza la instalación inicial de Windows Server y deja preparada una máquina base con los componentes necesarios para las fases posteriores.

La plantilla no se diseña como una imagen específica para un controlador de dominio, un servidor de ficheros o un cliente. Al contrario, se mantiene como una base genérica. Esta decisión reduce el mantenimiento: si es necesario actualizar componentes comunes, módulos DSC, herramientas de integración o configuraciones iniciales, basta con actualizar una única plantilla y no una plantilla distinta por cada rol.

Sysprep y Cloudbase-Init son elementos clave en este diseño. Sysprep elimina información específica de la máquina base antes del clonado, mientras que Cloudbase-Init permite que cada clon reciba parámetros propios durante el primer arranque. Con ello se combina una base común con una identidad individual para cada nodo.

4.1.4. Diseño del aprovisionamiento y parametrización inicial

El aprovisionamiento se diseña para que Terraform cree las máquinas virtuales a partir de la plantilla y asigne los parámetros de infraestructura necesarios. En esta ca-

pa se definen recursos como CPU, memoria, dirección IP, puerta de enlace, servidores DNS, discos y etiquetas de rol.

La parametrización inicial se delega en Cloudbase-Init. Terraform entrega metadatos mediante `configdrive2`, y Cloudbase-Init los aplica dentro del sistema operativo. Esta división permite mantener Terraform centrado en la infraestructura y evita usarlo para configurar en profundidad servicios internos de Windows.

En el servidor de ficheros, el diseño contempla un disco adicional destinado a datos. Esta asignación se realiza de forma condicional a partir de la etiqueta del nodo, de modo que solo el servidor de ficheros recibe almacenamiento extra. Así se evita duplicar bloques de código y se mantiene una definición común para los nodos del laboratorio.

4.1.5. Diseño de la configuración con DSC

La configuración mediante DSC se diseña en modo *push*. Las configuraciones se generan desde el equipo de administración (orquestador) y se aplican sobre los nodos Windows, que ya tienen preparado el Local Configuration Manager desde la imagen base.

El orden de aplicación de las configuraciones no es arbitrario. Primero se preparan y comprueban los mecanismos de administración remota; después se configura el controlador de dominio, ya que proporciona Active Directory y DNS; posteriormente se crean los objetos de dominio necesarios para las pruebas; a continuación se integran el servidor de ficheros y el cliente; y, por último, se aplican los recursos compartidos, permisos y comprobaciones funcionales. Esta secuencia evita que una configuración dependa de servicios que todavía no están disponibles.

El servidor de ficheros se divide en dos fases: unión al dominio y configuración de los recursos compartidos. Esta separación es necesaria porque los permisos NTFS y SMB dependen de grupos de Active Directory. Aplicarlos antes de que el servidor esté integrado en el dominio podría provocar errores de resolución de identidades o permisos incompletos.

4.1.6. Diseño de la validación y del bastionado

La validación se diseña en dos momentos. Primero se valida el entorno antes del bastionado para comprobar que el despliegue automatizado produce un laboratorio funcional. Después se aplican las guías CCN-STIC y se repiten las comprobaciones esenciales para verificar que el endurecimiento no rompe los servicios principales.

Esta validación doble permite separar dos tipos de problemas. Si una prueba falla antes del bastionado, el error se asocia al despliegue o a la configuración inicial. Si

una prueba funcionaba antes y falla después, el problema puede estar relacionado con las políticas de seguridad aplicadas. Esta distinción es importante para interpretar los resultados y evitar atribuir al bastionado errores que realmente pertenecen a fases anteriores.

El bastionado también se contempla desde el diseño mediante cuentas BreakGlass. Estas cuentas permiten conservar capacidad de administración cuando las políticas restringen cuentas integradas, reglas de firewall, grupos locales o mecanismos de acceso remoto.

4.2. Implementación del flujo automatizado

Una vez definido el diseño por capas, se implementa el flujo de automatización sobre el laboratorio. Esta sección describe cómo se materializa cada capa: primero se prepara una imagen base de Windows Server, después se genera una plantilla reutilizable, a continuación se despliegan los nodos mediante Terraform y, finalmente, se aplican las configuraciones DSC y las guías CCN-STIC.

Los listados completos de código se recogen en el Anexo A y en el **repositorio de GitHub del TFG**. En esta sección se conserva únicamente la explicación de los pasos ejecutados y la relación entre los artefactos utilizados.

4.2.1. Preparación de la imagen base y creación de la plantilla

La primera fase de implementación consiste en construir una base común de Windows Server que pueda reutilizarse para desplegar los nodos finales del laboratorio. Para ello se parte de una ISO original de Windows Server 2025, se incorporan los ficheros necesarios para la instalación desatendida y se genera una máquina base que posteriormente se convierte en plantilla de Proxmox.

Para preparar el archivo de respuesta se emplean herramientas incluidas en Windows Kits. Desde el orquestador, el **Administrador de Imágenes de Sistema de Windows** permite trabajar con el fichero `autounattend.xml` y comprobar que sus componentes son coherentes con la imagen de Windows utilizada. Una vez preparada la estructura de ficheros, el **Entorno de herramientas de implementación y creación de imágenes** permite generar la ISO final mediante `oscdimg`.

La relación entre `autounattend.xml` y la ISO es directa: el archivo se coloca en la raíz del medio de instalación personalizado. Al arrancar desde esa ISO, Windows Setup detecta automáticamente el fichero de respuesta y aplica la instalación desatendida.

La ISO personalizada se construye a partir de una copia del medio de instalación original. Sobre esta copia se añaden tres elementos principales: el archivo `autounattend.xml`, el script `Setup.ps1` y una carpeta de componentes auxiliares. El comando utilizado para generar la ISO se incluye en el Listado A.1 del Anexo A.1.

Dentro del archivo `autounattend.xml` se configuran varias fases de la instalación. En la fase inicial se definen parámetros como idioma, teclado, selección de imagen y particionado. En la fase `oobeSystem` se automatiza la experiencia inicial de Windows, evitando tener que completar manualmente el asistente de primer arranque.

También se configura el inicio automático temporal del usuario Administrador para ejecutar `Setup.ps1` tras la instalación. La contraseña utilizada en el fichero de respuesta aparece codificada en Base64, tal y como requiere el formato de respuesta de Windows, aunque esto no debe interpretarse como un mecanismo seguro de cifrado.

El script `Setup.ps1` se ejecuta automáticamente tras la instalación inicial, gracias al *AutoLogon* del usuario Administrador. Su función es copiar módulos DSC desde la ISO, preparar la administración remota, asignar una IP inicial, instalar Cloudbase-Init, incluir componentes de integración con Proxmox y configurar el Local Configuration Manager. La estructura funcional del script se recoge en el Listado A.2 del Anexo A.1.

Una parte importante de `Setup.ps1` es la preparación de la administración remota. Durante esta fase se habilitan WinRM, PowerShell Remoting, RDP y las reglas de firewall necesarias para poder operar sobre la máquina sin depender de la consola gráfica del hipervisor. WinRM se utiliza como vía principal para aplicar DSC en modo *push*, mientras que OpenSSH Server se mantiene como vía alternativa durante pruebas o situaciones de recuperación.

Otra tarea relevante de `Setup.ps1` es dejar configurado el Local Configuration Manager. En este trabajo se prepara el LCM para trabajar en modo *push*, de forma que las configuraciones DSC se generen desde el equipo de administración y se apliquen sobre los nodos cuando corresponda. Durante la preparación de la plantilla se utiliza un modo conservador, `ApplyOnly`, para aplicar la configuración necesaria sin activar correcciones automáticas mientras se actualiza, generaliza y convierte la VM en plantilla. Otros modos, como `ApplyAndMonitor` o `ApplyAndAutoCorrect`, resultan útiles cuando se desea detectar o corregir desviaciones de configuración, pero no eran necesarios en esta fase inicial.

La Tabla 4.3 resume los elementos incorporados en la imagen base.

Tabla 4.3: Elementos incorporados en la imagen base

Elemento	Función dentro del flujo
<code>autounattend.xml</code>	Automatiza la instalación de Windows Server, configura fases como <code>oobeSystem</code> y ejecuta el script de preparación inicial.
<code>Setup.ps1</code>	Prepara la máquina base: módulos DSC, administración remota, <code>Cloudbase-Init</code> , integración con Proxmox y LCM.
<code>Cloudbase-Init</code>	Permite que los clones reciban parámetros específicos en el primer arranque.
<code>Unattend.xml</code> de <code>Cloudbase-Init</code>	Se utiliza junto con Sysprep para preparar la plantilla antes del clonado.
QEMU Guest Agent y VirtIO	Mejoran la integración de Windows con Proxmox en red, almacenamiento y gestión.
Módulos DSC y scripts auxiliares	Reducen la dependencia de descargas externas durante fases críticas del despliegue.

También se copian los módulos DSC necesarios a la ruta estándar de Windows PowerShell. Esta decisión reduce la dependencia de repositorios externos durante el despliegue y permite que la futura plantilla contenga los recursos requeridos para configurar los nodos del laboratorio.

Una vez finalizado el script `Setup.ps1`, se aplica la configuración con DSC mediante el script `01-Base.ps1`. Este script prepara el sistema antes de convertirlo en plantilla, pero no instala roles como Active Directory o servidor de ficheros. Su función es dejar la base actualizada, comprobar que los componentes auxiliares están disponibles y asegurar que la máquina está lista para generalizarse. El fragmento más representativo se encuentra en el Listado A.3 del Anexo A.2.

El flujo completo de generación de la plantilla sigue una secuencia cerrada. Primero se sube la ISO personalizada a Proxmox. Después, Terraform crea la VM base y la arranca desde ese medio. Durante la instalación, Windows Setup lee `autounattend.xml` y ejecuta la instalación desatendida. Tras el primer inicio de sesión, `Setup.ps1` prepara la máquina con módulos, administración remota, `Cloudbase-Init` y componentes de integración. A continuación, se aplica `01-Base.ps1` para actualizar y verificar la base. Finalmente, se ejecuta Sysprep con el `Unattend.xml` de `Cloudbase-Init`.

Sysprep elimina información específica de la instalación original y deja el sistema preparado para ser clonado. El comando empleado se recoge en el Listado A.4 del Anexo A.2.

Cuando Sysprep finaliza, la máquina se apaga automáticamente. En ese estado se convierte en plantilla dentro de Proxmox. A partir de este momento, la plantilla contie-

ne una base común de Windows Server con los componentes necesarios ya integrados, pero sin una identidad definitiva asociada a un rol concreto.

La Tabla 4.4 resume el proceso seguido hasta obtener la plantilla reutilizable.

Tabla 4.4: Proceso de creación de la plantilla base reutilizable

Fase	Resultado esperado
Generación de ISO personalizada	Medio de instalación con archivo de respuesta, script de inicialización y componentes auxiliares.
Creación de VM base con Terraform	Máquina virtual inicial desplegada en Proxmox desde la ISO personalizada.
Instalación desatendida	Windows Server queda instalado sin intervención manual y ejecuta la preparación inicial.
Preparación interna	La VM base dispone de administración remota, Cloudbase-Init, módulos DSC y componentes de integración.
Configuración base con DSC	El script <code>01-Base.ps1</code> actualiza y comprueba la máquina antes de generalizarla.
Generalización con Sysprep	La instalación elimina información propia de la VM base y queda preparada para clonado.
Conversión en plantilla	La máquina se transforma en una plantilla genérica reutilizable para desplegar los nodos finales.

4.2.2. Aprovisionamiento con Terraform y Cloudbase-Init

Una vez creada la plantilla, Terraform se utiliza para desplegar los nodos finales del laboratorio. En esta fase se crean el controlador de dominio, el servidor de ficheros y el cliente Windows. Terraform define los recursos de infraestructura, mientras que Cloudbase-Init aplica en el interior de cada nodo la parametrización inicial recibida.

La configuración de Terraform se basa en una definición parametrizada de máquinas. Cada nodo se describe mediante un conjunto de atributos: identificador, nombre, dirección IP, puerta de enlace, memoria, número de núcleos y etiquetas. La definición completa de la variable de nodos y del recurso de clonado se recoge en los Listados A.5 y A.6 del Anexo A.3.

La interacción entre Terraform y Cloudbase-Init se realiza mediante `configdrive2`. Durante el clonado, Terraform genera un disco de inicialización asociado a la VM. En ese disco se almacenan metadatos de la instancia, como dirección IP, puerta de enlace y servidores DNS. Cloudbase-Init, instalado previamente en la plantilla, se ejecuta durante el primer arranque del clon, detecta el disco de metadatos y aplica esa configuración dentro de Windows.

Gracias a este mecanismo, la plantilla puede mantenerse genérica. No contiene una IP, un nombre o una configuración definitiva de nodo. Es en el primer arranque de cada clon cuando Cloudbase-Init transforma esa base común en una máquina con parámetros propios.

La Tabla 4.5 resume los parámetros aplicados por Terraform durante el aprovisionamiento. A diferencia de la tabla de nodos del Capítulo 3, aquí no se pretende repetir la descripción funcional del laboratorio, sino mostrar qué aspecto concreto del despliegue queda definido en la capa de infraestructura.

Tabla 4.5: Parámetros aplicados por Terraform durante el aprovisionamiento

Nodo	VMID	Parámetros de red	Recurso diferencial	Motivo técnico
DC1	100	IP fija y DNS inicial	Sin disco adicional	Debe quedar disponible como primer servicio de dominio y DNS.
FS2	110	IP fija apuntando al DNS del dominio	Disco adicional de datos	Necesita almacenamiento separado para recursos compartidos.
CLI1	120	IP fija apuntando al DNS del dominio	Sin disco adicional	Se utiliza como cliente de validación unido al dominio.

En el caso del servidor de ficheros, Terraform añade un disco adicional destinado al almacenamiento de datos. Esta asignación se realiza de forma condicional, en función de las etiquetas del nodo, como se muestra en el Listado A.7 del Anexo A.3. De este modo, la infraestructura se adapta al rol sin duplicar bloques completos de configuración.

Al finalizar esta fase, las máquinas ya existen en Proxmox y disponen de parámetros iniciales propios. Sin embargo, todavía no desempeñan sus roles funcionales. Esa transformación se realiza en la siguiente fase mediante PowerShell DSC.

4.2.3. Configuración de los nodos mediante PowerShell DSC

La configuración declarativa mediante PowerShell DSC transforma las máquinas desplegadas desde la plantilla en nodos funcionales del laboratorio. En esta fase se aplica el estado deseado de cada sistema: roles instalados, servicios activos, pertenencia al dominio, usuarios, grupos, carpetas, recursos compartidos y permisos.

Como ya hemos visto, el modelo utilizado es *push*. Las configuraciones se generan desde el orquestador y se aplican sobre los nodos de destino, que ya tienen configurado el Local Configuration Manager. Para facilitar la depuración, el despliegue se divide en configuraciones separadas por responsabilidad.

La Tabla 4.6 resume los principales bloques DSC utilizados.

Tabla 4.6: Bloques principales de configuración DSC

Bloque	Implementación	Función principal
Comprobación común	WinRM, SSH, LCM y conectividad	Verificar que los nodos son administrables antes de aplicar configuraciones específicas.
Controlador de dominio	AD01-Promo-DC-Config.ps1, AD02-OUObjectsV2.ps1	Instalar AD DS y DNS, crear el bosque y generar la estructura inicial de Active Directory.
Servidor de ficheros	FS-JoinDomain.ps1, FS-Shares.ps1	Unir el servidor al dominio, preparar almacenamiento, publicar recursos SMB y aplicar permisos.
Cliente Windows	CLI-Base.ps1	Unir el equipo al dominio y utilizarlo como nodo de validación funcional.
BreakGlass	Cuentas locales y cuenta de dominio	Mantener vías de administración alternativas antes y después del bastionado.

Los scripts con el código más relevante se recogen en los Anexos A.4, A.5 y A.6.

El controlador de dominio es el primer nodo configurado, ya que el resto del entorno depende de Active Directory y DNS. Su configuración se divide en dos partes. La primera instala las características necesarias, configura la dirección DNS del propio servidor y promueve el nodo como controlador de dominio del bosque `ad.umtfg.com`. Esta fase utiliza recursos como `WindowsFeature`, `DnsServerAddress`, `ADDomain` y `DnsServerForwarder`. El bloque representativo se recoge en el Listado A.8 del Anexo A.4.

La segunda parte crea la estructura lógica de Active Directory: unidades organizativas, usuarios, grupos de seguridad y la cuenta `BreakGlass` de dominio. Estos objetos no solo organizan el laboratorio, sino que permiten que las pruebas funcionales sean repetibles, ya que los usuarios y grupos empleados para validar permisos son generados por el propio flujo automatizado. La cuenta `BreakGlass` de dominio también se crea en esta fase porque depende de que Active Directory ya esté disponible. Esta parte se recoge en el Listado A.9 del Anexo A.4.

La configuración del servidor de ficheros se divide en dos fases por dependencia técnica. Primero se une el servidor al dominio; después, una vez reiniciado e integrado en Active Directory, se configuran el almacenamiento, el rol de ficheros, los recursos SMB y los permisos. Antes de unir el servidor al dominio se comprueba la disponibilidad de los registros SRV de Active Directory. Esta comprobación evita iniciar el `join` si el controlador de dominio o DNS todavía no están listos. El bloque correspondiente se

recoge en el Listado A.10 del Anexo A.5.

Tras la unión al dominio, se instala el rol **FS-FileServer**, se asegura el servicio **LanmanServer**, se prepara el volumen de datos y se crea el recurso compartido principal. También se activa Access-Based Enumeration para que los usuarios solo vean los recursos a los que tienen acceso. La configuración se recoge en el Listado A.11 del Anexo A.5.

El modelo de permisos se basa en grupos de Active Directory, separando permisos SMB y NTFS. De esta forma, los permisos no se asignan directamente a usuarios individuales, sino a grupos de lectura o modificación. Esta decisión facilita el mantenimiento del modelo de acceso y permite validar tanto accesos permitidos como denegaciones. El esquema de permisos se incluye en el Listado A.12 del Anexo A.5.

El cliente Windows se configura para actuar como nodo de validación. Aunque no desempeña un rol de servidor, resulta necesario para comprobar que el dominio, DNS, los recursos compartidos y los permisos funcionan desde la perspectiva de un equipo final unido al dominio.

La configuración del cliente incluye varias tareas. Primero se crea una cuenta Break-Glass local para conservar una vía de acceso administrativa incluso si la comunicación con el dominio falla. A continuación, se aplican ajustes de red, entre ellos la desactivación de IPv6 en el adaptador del laboratorio. Esta decisión se adopta para reducir ambigüedades durante las pruebas de resolución de nombres y asegurar que la validación se realiza sobre la configuración IPv4 definida en Terraform y Cloudbase-Init.

También se fuerza el registro DNS del cliente dentro del dominio mediante las opciones del adaptador de red. Esta comprobación es importante porque el cliente no solo debe unirse al dominio, sino aparecer correctamente registrado y resolverse mediante DNS interno. Después se ejecuta la unión al dominio en la OU correspondiente y se realiza el reinicio necesario para completar el proceso.

Por último, se controla la pertenencia de los grupos administrativos definidos en el dominio dentro del grupo local *Administradores*. Esta parte permite que las cuentas de administración previstas, incluida la cuenta BreakGlass de dominio cuando corresponda, puedan gestionar el cliente. Tras el reinicio producido por el *join*, se comprueba de nuevo la administración remota, ya que la pertenencia al dominio y las políticas aplicadas posteriormente pueden modificar perfiles de red, reglas de firewall o permisos de acceso remoto. El bloque principal se recoge en el Listado A.13 del Anexo A.6.

4.2.4. Aplicación automatizada de guías CCN-STIC

Tras validar el funcionamiento básico del entorno, se aplica una fase de bastionado basada en guías CCN-STIC. El objetivo no es certificar formalmente el cumplimiento del entorno, sino comprobar que las políticas de endurecimiento pueden integrarse en el flujo automatizado sin romper la funcionalidad esencial del laboratorio.

En esta fase se utilizan tres guías principales: CCN-STIC-570A25 para sistemas Windows Server, CCN-STIC-573-25 para el servidor de ficheros y CCN-STIC-599AB23 para el cliente Windows. Las configuraciones se aplican mediante copias de seguridad de GPO importadas en el dominio y enlazadas a las unidades organizativas correspondientes.

La aplicación de las guías se centraliza en el controlador de dominio, ya que desde este nodo se importan y enlazan las GPO. Primero se copian los paquetes ZIP y los scripts auxiliares al directorio de trabajo `C:\CCN`. Esta fase de *staging* crea una estructura de carpetas para paquetes, scripts, logs, estado, extracción temporal y trabajo intermedio. Con ello se evita ejecutar las guías desde rutas externas o dependientes del equipo de administración. El procedimiento de copia se recoge en el Listado A.14 del Anexo A.7.

A continuación, se ejecuta una configuración DSC que instala las herramientas de administración de GPO, comprueba la existencia de los directorios de trabajo e invoca los scripts de aplicación de las guías. Cada bloque utiliza marcas de estado para evitar reaplicaciones innecesarias. La estructura de esta configuración se recoge en el Listado A.15 del Anexo A.7, y el comando de ejecución en el Listado A.16.

La Tabla 4.7 resume las guías aplicadas.

Tabla 4.7: Guías CCN-STIC utilizadas en el laboratorio

Guía	Ámbito	Nivel utilizado	Destino principal
CCN-STIC-570A25	Windows Server	ENS	OU de servidores y políticas base.
CCN-STIC-573-25	Servidor de ficheros	ESTANDAR	OU de servidores de ficheros.
CCN-STIC-599AB23	Cliente Windows	ESTANDAR	OU de estaciones de trabajo.

El procedimiento permite importar inicialmente las GPO con enlaces deshabilitados, comprobar su creación y activarlas posteriormente sobre las unidades organizativas correspondientes. En la validación final, los enlaces deben estar habilitados para comprobar el impacto real de las políticas.

4.3. Validación funcional y comprobación tras bastionado

La validación del entorno no se plantea como una comprobación aislada de comandos, sino como un escenario funcional completo. El objetivo es verificar que las fases anteriores han producido una infraestructura Windows operativa, integrada y administrable. Para ello se comprueba el comportamiento del dominio, la resolución DNS, la unión de los nodos, la aplicación de configuraciones DSC, el acceso a recursos compartidos, la denegación de accesos no autorizados y la continuidad administrativa.

El escenario de validación se ejecuta en dos momentos. Primero se valida el entorno antes de aplicar bastionado, con el fin de comprobar que la infraestructura desplegada mediante Terraform y DSC funciona correctamente. Después se aplican las guías CCN-STIC y se repiten las comprobaciones esenciales para verificar que el endurecimiento no rompe los servicios principales ni bloquea la administración.

Los comandos concretos utilizados para estas comprobaciones se recogen en el Anexo A.8. En este apartado se muestran las pruebas y resultados, no la ejecución detallada de cada comando.

4.3.1. Escenario de validación

Como se muestra en la Figura 4.1, la validación se articula alrededor de tres elementos: el cliente Windows unido al dominio, el controlador de dominio y el servidor de ficheros. Desde el cliente se comprueba que la resolución DNS apunta correctamente al controlador de dominio, que la autenticación se realiza contra Active Directory y que el acceso al recurso compartido del servidor de ficheros respeta el modelo de permisos basado en grupos.

El caso de uso incluye varios perfiles de usuario para comprobar comportamientos distintos. Un usuario con permisos de modificación debe poder acceder y escribir sobre su carpeta departamental; un usuario con permisos limitados solo debe acceder a los recursos públicos; y un usuario sin pertenencia a grupos de acceso debe recibir una denegación al intentar acceder a recursos restringidos. Además, se valida una cuenta administrativa BreakGlass para comprobar que la administración remota sigue siendo posible antes y después del bastionado.

4.3.2. Caso de uso de acceso a recursos compartidos

La validación se realiza desde el cliente Windows unido al dominio y se apoya en los usuarios creados previamente mediante DSC. Esto permite comprobar el modelo de permisos sin introducir usuarios o grupos manuales después del despliegue.

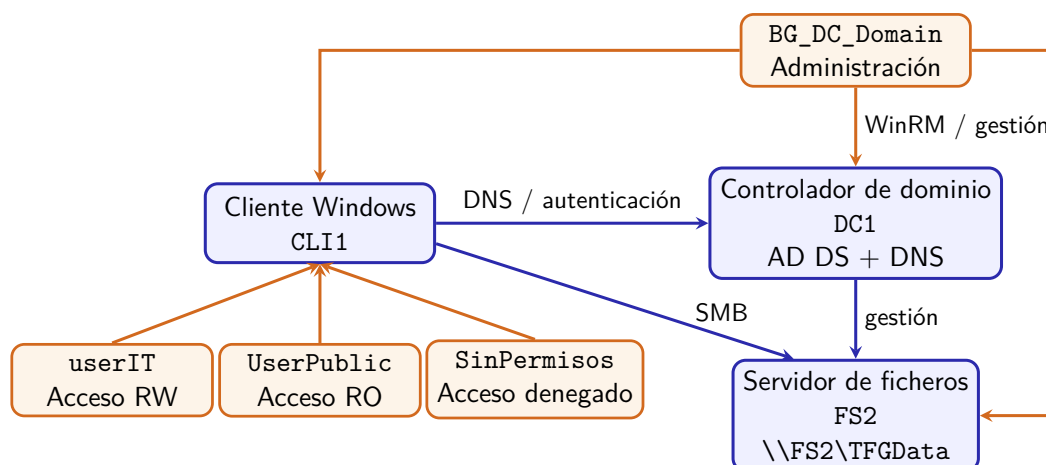


Figura 4.1: Escenario de validación funcional antes y después del bastionado.

La Tabla 4.8 resume el escenario funcional planteado.

Tabla 4.8: Escenario funcional de validación basado en perfiles de usuario

Usuario	Perfil	Acción validada	Resultado esperado
userIT	Usuario del área IT	Acceso a \\FS2\TFGData\IT y escritura de prueba.	Acceso permitido con modificación.
userFIN	Usuario de Finanzas	Acceso a recursos financieros y proyectos compartidos asignados.	Acceso permitido según pertenencia a grupos.
userRRHH	Usuario de RRHH	Acceso a carpeta propia y prueba de acceso a carpeta no asignada.	Acceso permitido al recurso propio y denegado al no asignado.
UserPublic	Usuario con acceso limitado	Acceso a recursos públicos sin permisos de modificación sobre carpetas restringidas.	Acceso de lectura a recursos públicos.
SinPermisos	Usuario sin grupos de acceso	Intento de acceso a carpetas departamentales y proyectos restringidos.	Acceso denegado.
BG_DC_Domain	Usuario administrativo	Creación de sesión remota y comprobación de administración tras el bastionado.	Administración disponible mediante cuenta prevista.

Este escenario es más amplio que una simple prueba de conectividad. La validación comprueba simultáneamente varios aspectos del entorno: que el cliente se comunica con el dominio, que DNS resuelve los nombres internos, que los usuarios y grupos creados mediante DSC existen en Active Directory, que el servidor de ficheros aplica permisos SMB y NTFS coherentes, que Access-Based Enumeration limita la visibilidad de carpetas y que la administración remota no queda bloqueada tras aplicar políticas de seguridad.

4.3.3. Resultados antes del bastionado

La Tabla 4.9 resume las comprobaciones realizadas antes de aplicar las guías CCN-STIC.

Tabla 4.9: Resultados de validación funcional antes del bastionado

Código	Prueba	Evidencia obtenida	Resultado
VF-01	Dominio operativo	El dominio <code>ad.umtfg.com</code> responde desde el controlador de dominio.	Superada
VF-02	Resolución DNS interna	Los nodos resuelven el dominio y el controlador de dominio mediante DNS interno.	Superada
VF-03	Unión de nodos al dominio	El servidor de ficheros y el cliente aparecen en Active Directory dentro de sus OU.	Superada
VF-04	Estado de DSC	La última configuración aplicada muestra estado correcto en los nodos.	Superada
VF-05	Acceso permitido	Un usuario autorizado accede al recurso compartido correspondiente.	Superada
VF-06	Acceso denegado	Un usuario sin permisos no accede a recursos restringidos.	Superada
VF-07	Administración remota	La cuenta administrativa prevista permite gestionar nodos críticos.	Superada

Las evidencias de la Tabla 4.9 se obtuvieron mediante los comandos de comprobación recogidos en el Anexo A.8. En particular, se emplearon consultas sobre Active Directory, resolución DNS, estado de DSC, recursos SMB, permisos NTFS y pruebas de acceso desde el cliente. De este modo, la validación comprueba el comportamiento funcional del entorno desplegado antes del bastionado.

Esta validación previa es necesaria para interpretar correctamente la fase posterior: si el laboratorio no funcionara antes de aplicar las guías, no sería posible atribuir los fallos posteriores al endurecimiento.

4.3.4. Resultados después del bastionado

Tras aplicar las guías CCN-STIC, se repiten las comprobaciones esenciales. Los comandos utilizados se encuentran en el Anexo A.8. La Tabla 4.10 resume los resultados.

Tabla 4.10: Resultados de validación después del bastionado

Código	Comprobación	Resultado	Observación
CCN-01	Disponibilidad del dominio	Superada	Active Directory continúa prestando los servicios esenciales.
CCN-02	Resolución de nombres	Superada	Los nodos siguen resolviendo nombres internos del dominio.
CCN-03	Acceso a recursos compartidos	Superada	Los usuarios autorizados conservan acceso a los recursos previstos.
CCN-04	Control de permisos	Superada	Los usuarios no autorizados continúan sin acceso a recursos restringidos.
CCN-05	Administración remota	Superada con ajuste	La administración se conserva mediante cuentas y excepciones previstas.
CCN-06	Acceso BreakGlass	Superada	Las cuentas alternativas mantienen la capacidad administrativa definida.

La comprobación posterior al bastionado se centró en los servicios esenciales del laboratorio, no en una auditoría completa de cumplimiento. Por ello, las pruebas se orientaron a verificar que Active Directory, DNS, los recursos compartidos, el modelo de permisos y los mecanismos de administración seguían disponibles tras la aplicación de las GPO. Esta validación permite confirmar la compatibilidad operativa del bastionado con el flujo automatizado, aunque no sustituye a una revisión formal de seguridad.

4.4. Discusión y análisis de resultados

Una vez completadas las fases de diseño, implementación y validación, resulta necesario interpretar los resultados obtenidos en conjunto. Las secciones anteriores han descrito cómo se construye el entorno y qué pruebas se han ejecutado; esta sección analiza qué valor aportan esos resultados respecto al objetivo del trabajo y qué aspectos técnicos han condicionado la solución final.

El análisis no se centra en rendimiento, concurrencia o carga, ya que esos aspectos quedan fuera del alcance definido. La discusión se orienta a valorar si el flujo propuesto permite construir un entorno Windows reproducible, funcional, validable y compatible con una fase posterior de bastionado.

4.4.1. Análisis del enfoque por capas

El principal resultado técnico del trabajo es que la separación por capas ha permitido ordenar un proceso que, ejecutado manualmente, combina tareas muy distintas: instalación del sistema operativo, preparación de una plantilla, creación de máquinas

virtuales, parametrización inicial, configuración de roles Windows, aplicación de permisos y validación posterior.

La división entre Terraform, Cloudbase-Init y DSC ha sido especialmente relevante. Terraform ha quedado limitado a la creación de infraestructura virtual, evitando que asuma tareas internas del sistema operativo. Cloudbase-Init ha resuelto la parametrización inicial de los clones, permitiendo reutilizar una plantilla genérica sin renunciar a que cada nodo tenga identidad y configuración de red propia. DSC, por su parte, ha asumido la configuración funcional de Windows, donde resulta adecuado por su integración con PowerShell, Active Directory, DNS, servicios Windows, recursos SMB y permisos NTFS.

Esta separación ha facilitado la depuración del entorno. Cuando una fase falla, el problema puede localizarse en una capa concreta: imagen base, clonado, parametrización, configuración declarativa, bastionado o validación. Esta propiedad es importante porque el laboratorio incluye dependencias fuertes entre componentes. El servidor de ficheros no puede aplicar permisos basados en grupos si antes no existe el dominio; el cliente no puede validarse si DNS no resuelve correctamente; y el bastionado no puede interpretarse si antes no se ha comprobado el estado funcional inicial.

4.4.2. Análisis de la validación funcional

La validación funcional confirma que el entorno generado no se limita a contener máquinas virtuales encendidas, sino que proporciona servicios integrados. El controlador de dominio ofrece Active Directory y DNS; el servidor de ficheros se une al dominio, publica recursos SMB y aplica permisos; y el cliente permite comprobar el comportamiento desde la perspectiva de un equipo final.

El caso de uso basado en perfiles de usuario aporta valor porque valida tanto accesos permitidos como accesos denegados. Esta doble comprobación es necesaria en un servidor de ficheros. Un resultado correcto no consiste únicamente en que un usuario pueda abrir una carpeta, sino en que cada usuario solo acceda a los recursos que le corresponden según su pertenencia a grupos. Por ello, las pruebas con usuarios de IT, Finanzas, RRHH, acceso público y usuario sin permisos permiten comprobar el modelo de control de acceso desde varios ángulos.

Además, la validación desde el cliente demuestra que las configuraciones aplicadas en el dominio y en el servidor de ficheros tienen efecto real sobre un equipo consumidor del servicio. Esto evita validar únicamente desde el propio servidor, donde algunas comprobaciones podrían resultar menos representativas del uso real del entorno.

4.4.3. Análisis del bastionado y de la continuidad administrativa

La aplicación de guías CCN-STIC añade una capa de complejidad al flujo de automatización. Las políticas de bastionado pueden modificar cuentas por defecto, privilegios locales, reglas de firewall o mecanismos de administración remota. Por este motivo, la validación posterior al bastionado no se ha planteado como una auditoría formal de cumplimiento, sino como una comprobación de compatibilidad operativa: el entorno debe seguir prestando sus servicios principales y debe seguir siendo administrable.

Los resultados posteriores al bastionado muestran que los servicios esenciales se mantienen disponibles cuando la continuidad administrativa se prepara antes de aplicar las políticas. La existencia de cuentas BreakGlass locales y de dominio reduce el riesgo de perder acceso a los nodos tras deshabilitar o restringir cuentas integradas. Esta decisión resulta especialmente importante porque un entorno endurecido pero no administrable no puede considerarse operativo desde el punto de vista de la gestión de sistemas.

El ajuste necesario en administración remota también es un resultado relevante. No debe interpretarse como un fallo del flujo, sino como una evidencia de que el bastionado afecta a elementos operativos que deben contemplarse desde el diseño. Por tanto, la automatización no puede limitarse a desplegar y configurar servicios; también debe prever cómo se administrarán esos servicios después de aplicar políticas restrictivas.

4.4.4. Incidencias técnicas y decisiones adoptadas

Durante el desarrollo se identificaron incidencias relacionadas con reinicios, disponibilidad de DNS, unión al dominio, resolución de grupos, clonado de máquinas y administración remota. Estas incidencias no se trataron como errores aislados, sino como señales para ajustar el diseño final.

La Tabla 4.11 resume las principales incidencias y las decisiones adoptadas.

Tabla 4.11: Incidencias técnicas y decisiones de diseño adoptadas

Aspecto	Problema o riesgo	Decisión adoptada
Flujo completo	Un único bloque dificulta localizar errores y gestionar reinicios.	Dividir el despliegue en fases independientes y comprobables.
Plantilla base	Mantener varias plantillas por rol aumenta el trabajo de actualización.	Crear una única plantilla genérica y aplicar roles mediante DSC.
Clonado	Los clones no deben conservar identidad de la máquina base.	Generalizar con Sysprep y parametrizar el primer arranque con Cloudbase-Init.
Dominio y DNS	El servidor de ficheros y el cliente dependen de la disponibilidad del DC.	Comprobar registros SRV y resolución DNS antes de unir equipos al dominio.

Continúa en la siguiente página →

Aspecto	Problema o riesgo	Decisión adoptada
Permisos del servidor de ficheros	Los grupos de dominio no pueden resolverse antes del <i>join</i> .	Separar unión al dominio y configuración posterior de recursos compartidos.
Administración remota	WinRM puede verse afectado por red, dominio o políticas de seguridad.	Preparar administración remota antes del bastionado y mantener cuentas alternativas.
Bastionado	Las GPO pueden restringir cuentas, firewall o privilegios.	Validar antes y después de CCN-STIC y usar cuentas BreakGlass.

Estas decisiones muestran que la automatización no consiste únicamente en ejecutar scripts, sino en ordenar dependencias, reinicios, validaciones y mecanismos de recuperación. El resultado es un flujo más sencillo de repetir y más fácil de depurar, aunque requiera dividir el proceso en varias fases.

4.4.5. Valoración global de los resultados

La valoración global de los resultados es positiva. El flujo implementado permite pasar de una ISO original de Windows Server a un entorno compuesto por controlador de dominio, servidor de ficheros y cliente Windows, con configuración automatizada y validación funcional. La solución no elimina toda la complejidad del despliegue, pero la desplaza desde acciones manuales dispersas hacia artefactos controlados: ficheros de respuesta, scripts, configuraciones Terraform, configuraciones DSC, GPO importadas y comandos de validación.

El valor principal del trabajo está en la integración de esas piezas. Cada herramienta resuelve una parte concreta del problema, pero el resultado solo es útil cuando se coordina el orden de ejecución, las dependencias entre nodos, los reinicios, los permisos y las comprobaciones posteriores. En ese sentido, el proyecto demuestra que la automatización de sistemas Windows no debe evaluarse únicamente por la creación de máquinas virtuales, sino por la capacidad de entregar un entorno funcional y administrable.

La validación antes y después del bastionado refuerza esta conclusión. Primero se comprueba que el entorno funciona sin políticas restrictivas adicionales; después se verifica que los servicios esenciales continúan operativos tras aplicar guías CCN-STIC. Esta doble comprobación permite separar los errores propios del despliegue de los posibles efectos introducidos por el endurecimiento.

Finalmente, los resultados muestran que la continuidad administrativa debe formar parte del diseño inicial. La creación de cuentas BreakGlass y la comprobación de administración remota no son elementos accesorios, sino condiciones necesarias para que el entorno pueda seguir gestionándose después del bastionado. Esta idea constituye una de

las aportaciones prácticas más relevantes del trabajo, ya que conecta la automatización del despliegue con la operación posterior de la infraestructura.

5. Conclusiones y vías futuras

Este capítulo recoge las conclusiones generales del Trabajo Fin de Grado, las limitaciones principales del entorno desarrollado y las posibles líneas de evolución futura. A diferencia del Capítulo 4, donde se han descrito el diseño, la implementación, la validación y el análisis técnico de los resultados, este capítulo se centra en valorar el cumplimiento de los objetivos planteados y en cerrar el alcance del trabajo.

5.1. Conclusiones

El desarrollo de este Trabajo Fin de Grado ha permitido diseñar, implementar y validar un flujo automatizado para desplegar y configurar un entorno Windows virtualizado con roles diferenciados. La solución construida demuestra que es posible combinar una imagen base personalizada, una plantilla reutilizable, aprovisionamiento mediante Terraform, parametrización inicial con Cloudbase-Init, configuración declarativa con PowerShell DSC y una fase posterior de bastionado basada en guías CCN-STIC.

El resultado obtenido no se limita a la creación automática de máquinas virtuales. El flujo permite construir un entorno funcional compuesto por un controlador de dominio, un servidor de ficheros y un cliente Windows unido al dominio. Además, el entorno se ha validado antes y después del bastionado, comprobando que los servicios principales siguen operativos y que la administración se conserva mediante las cuentas y excepciones previstas.

La Tabla 5.1 resume la relación entre los objetivos específicos definidos en la introducción y las evidencias obtenidas durante el desarrollo.

Tabla 5.1: Relación entre objetivos específicos y evidencias obtenidas

Objetivo	Descripción	Evidencia principal
OE-01	Diseñar una imagen base automatizada de Windows Server y generar una plantilla reutilizable en Proxmox.	ISO personalizada, instalación desatendida, ejecución de <code>Setup.ps1</code> , configuración base con DSC, Sysprep y conversión en plantilla.
OE-02	Automatizar el aprovisionamiento de la infraestructura virtual mediante Terraform.	Creación de la VM base y despliegue de DC1, FS2 y CLI1 desde la plantilla, con parámetros de red, recursos y disco adicional cuando corresponde.

Continúa en la siguiente página →

Objetivo	Descripción	Evidencia principal
OE-03	Definir y aplicar configuraciones declarativas mediante PowerShell DSC para los nodos del laboratorio.	Configuración del controlador de dominio, servidor de ficheros y cliente Windows mediante bloques DSC separados por rol y dependencia.
OE-04	Incorporar una fase de bastionado basada en guías CCN-STIC, manteniendo la continuidad administrativa.	Aplicación de guías CCN-STIC, preparación de cuentas BreakGlass y comprobación posterior de administración remota.
OE-05	Validar el funcionamiento del entorno antes y después del bastionado.	Pruebas sobre dominio, DNS, unión de nodos, estado de DSC, recursos compartidos, permisos, accesos denegados y administración remota.

Desde el punto de vista técnico, el trabajo confirma la utilidad de separar las responsabilidades entre herramientas. Terraform resulta adecuado para describir y crear la infraestructura virtual, mientras que PowerShell DSC permite configurar el estado interno de los sistemas Windows. Cloudbase-Init actúa como punto intermedio, ya que permite que los clones desplegados desde una plantilla común reciban parámetros propios durante el primer arranque.

También se confirma que PowerShell DSC 1.1 sigue siendo una tecnología válida para un escenario centrado en Windows Server y roles clásicos de infraestructura. Su integración con PowerShell, Windows Server, Active Directory, DNS, SMB y NTFS ha permitido expresar como configuración declarativa tareas que, de forma manual, implicarían numerosos pasos dependientes entre sí.

Otro resultado relevante es la importancia de validar el entorno desde la perspectiva de un cliente unido al dominio. La comprobación del dominio, DNS, recursos compartidos y permisos desde el propio cliente aporta una evidencia más realista que una validación limitada al servidor. Además, el uso de pruebas positivas y negativas ha permitido comprobar no solo que los usuarios autorizados acceden a los recursos previstos, sino también que los usuarios sin permisos no acceden a carpetas restringidas.

La integración del bastionado dentro del flujo de trabajo constituye otra conclusión importante. Las políticas de seguridad no deben aplicarse como una fase aislada sin considerar su impacto operativo. En este trabajo, la preparación de cuentas BreakGlass y la comprobación de administración remota han permitido mantener la capacidad de gestión tras aplicar las guías CCN-STIC. Por tanto, la automatización del despliegue debe contemplar no solo la creación y configuración de sistemas, sino también su operación posterior.

En conjunto, puede concluirse que la solución desarrollada cumple el objetivo general

del trabajo. El laboratorio resultante es reproducible, modular y validable dentro del alcance definido. Además, proporciona una base técnica sobre la que podrían añadirse nuevos roles, nuevas pruebas y mecanismos de seguridad más avanzados.

5.2. Limitaciones del trabajo

Aunque los resultados obtenidos son satisfactorios dentro del alcance planteado, el trabajo presenta una serie de limitaciones que deben tenerse en cuenta al interpretar sus conclusiones.

La primera limitación está relacionada con el entorno utilizado. El proyecto se ha desarrollado en un laboratorio virtualizado y controlado, compuesto por un número reducido de nodos. Esta escala permite validar el flujo de automatización de extremo a extremo, pero no representa por completo una infraestructura empresarial en producción.

El laboratorio se limita a tres roles principales: controlador de dominio, servidor de ficheros y cliente Windows. Esta selección es suficiente para comprobar identidad, resolución de nombres, unión al dominio, recursos compartidos, permisos y acceso desde cliente, pero deja fuera otros servicios habituales en entornos Windows más amplios, como WSUS, PKI corporativa, servidores de aplicaciones, copias de seguridad, monitorización o despliegue centralizado de software.

Tampoco se han abordado aspectos propios de arquitecturas de alta disponibilidad o gran escala. No se implementan varios controladores de dominio, replicación entre sedes, clústeres de ficheros, tolerancia a fallos, recuperación ante desastres ni segmentación avanzada de red. Estos elementos quedan fuera del alcance del TFG, aunque la arquitectura por capas permitiría incorporarlos en una evolución posterior.

La fase de bastionado también debe interpretarse dentro de los límites del trabajo. Las guías CCN-STIC se han utilizado como referencia técnica para aplicar configuraciones de seguridad y comprobar su impacto operativo, pero no se ha realizado una auditoría formal de cumplimiento del Esquema Nacional de Seguridad ni una certificación de la infraestructura resultante.

Otra limitación se encuentra en la gestión de credenciales. En el laboratorio se han empleado mecanismos suficientes para ejecutar y validar la automatización, evitando incluir contraseñas reales en el documento. Sin embargo, no se ha integrado una solución empresarial de gestión de secretos, rotación, custodia y auditoría de credenciales administrativas.

Por último, la validación realizada se centra en pruebas funcionales y operativas.

Se ha comprobado que el entorno se despliega, se configura y mantiene sus servicios principales antes y después del bastionado, pero no se han evaluado métricas de rendimiento, carga concurrente, resiliencia ante fallos o escalabilidad con un número elevado de nodos.

5.3. Vías futuras

A partir del trabajo desarrollado, se abren varias líneas de evolución. La primera consiste en ampliar el laboratorio con nuevos roles Windows, como WSUS, PKI, IIS, servicios de backup o monitorización centralizada. Esta ampliación permitiría comprobar si la arquitectura por capas sigue siendo mantenible en un entorno más completo.

Una segunda línea de trabajo consiste en reforzar la validación automática. Los comandos incluidos en el anexo podrían transformarse en una batería de pruebas ejecutable de forma repetible, por ejemplo mediante Pester u otra herramienta de validación. Esto permitiría generar informes automáticos sobre dominio, DNS, GPO, recursos compartidos, permisos y administración remota.

Otra evolución relevante sería mejorar la gestión de secretos. En una versión más avanzada del laboratorio, las credenciales utilizadas durante el despliegue deberían integrarse en un almacén seguro, con control de acceso, rotación periódica y registro de uso. Esta mejora resultaría especialmente importante para trasladar el enfoque a un entorno más cercano a producción.

Desde el punto de vista de la automatización, queda abierta la comparación del enfoque actual con alternativas como DSC 3.0 o Ansible. Esta comparación permitiría evaluar si el modelo clásico basado en PowerShell DSC 1.1 sigue siendo la opción más adecuada cuando el entorno crece, incorpora sistemas heterogéneos o requiere una orquestación más amplia.

La fase de bastionado también admite una ampliación mediante una validación de seguridad más completa. Esta línea podría incluir revisión de GPO aplicadas, análisis de cumplimiento, evaluación de excepciones, comprobación de configuraciones efectivas y ejecución de pruebas específicas sobre servicios expuestos.

Por último, el laboratorio podría evolucionar hacia una plataforma docente o de experimentación reutilizable. Su estructura permite reconstruir un entorno Windows desde cero y validar su funcionamiento, por lo que podría servir como base para prácticas de administración de sistemas, automatización, configuración segura y bastionado de infraestructuras virtualizadas.

Bibliografía

- [1] Microsoft Learn. Active directory domain services overview, 2023. URL <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/active-directory-domain-services>.
- [2] Microsoft Learn. File server and smb overview, 2023. URL <https://learn.microsoft.com/en-us/windows-server/storage/file-server/file-server-smb-overview>.
- [3] Kief Morris. *Infrastructure as Code: Managing Servers in the Cloud*. O'Reilly Media, 2016.
- [4] Akond Rahman, Khajeh Mahdavi-Hezaveh, and Laurie Williams. A mapping study on infrastructure as code. *Information and Software Technology*, 114:1–11, 2019.
- [5] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.
- [6] Kelley Johnson, Betsy Dempsey, Ron Ross, Sarbari Gupta, and Denis Bailey. Guide for security-focused configuration management of information systems. Technical Report SP 800-128, National Institute of Standards and Technology (NIST), 2011.
- [7] Microsoft Security Guidance. Windows security baselines, 2024. URL <https://learn.microsoft.com/en-us/windows/security/threat-protection/windows-security-baselines>.
- [8] Microsoft Learn. Smbv1 is not installed by default in windows server and windows, 2025. URL <https://learn.microsoft.com/en-us/windows-server/storage/file-server/troubleshoot/smbv1-not-installed-by-default-in-windows>.
- [9] Boletín Oficial del Estado. Real decreto 311/2022, de 3 de mayo, por el que se regula el esquema nacional de seguridad. BOE núm. 106, de 4 de mayo de 2022, 2022. URL <https://www.boe.es/buscar/act.php?id=BOE-A-2022-7191>.
- [10] CCN-CERT. Guías ccn-stic de acceso público, 2026. URL <https://www.ccn-cert.cni.es/es/series-ccn-stic/guias-de-acceso-publico-ccn-stic>.

- [11] HashiCorp. Introduction to terraform, 2024. URL <https://developer.hashicorp.com/terraform/intro>.
 - [12] HashiCorp. Terraform state, 2024. URL <https://developer.hashicorp.com/terraform/language/state>.
 - [13] Microsoft Learn. Powershell desired state configuration overview, 2023. URL <https://learn.microsoft.com/en-us/powershell/scripting/dsc/overview/overview>.
 - [14] Microsoft Learn. Configuring the local configuration manager (lcm), 2023. URL <https://learn.microsoft.com/en-us/powershell/scripting/dsc/managing-nodes/metaconfig>.
 - [15] Microsoft Learn. Powershell desired state configuration resources, 2023. URL <https://learn.microsoft.com/en-us/powershell/scripting/dsc/resources/resources>.
 - [16] Microsoft Learn. Enacting configurations (push and pull models), 2023. URL <https://learn.microsoft.com/en-us/powershell/scripting/dsc/managing-nodes/enactconfigs>.
 - [17] Microsoft Learn. Powershell desired state configuration v2.0 updates, 2022. URL <https://learn.microsoft.com/en-us/powershell/scripting/dsc/overview/dscv2>.
 - [18] Ansible Documentation. Introduction to ansible for windows, 2024. URL https://docs.ansible.com/ansible/latest/os_guide/windows_usage.html.
 - [19] Ansible Documentation. Setting up a windows host for ansible via winrm, 2024. URL https://docs.ansible.com/ansible/latest/os_guide/windows_winrm.html.
 - [20] Ansible Documentation. Using powershell dsc resources in ansible (win_dsc module), 2023. URL https://docs.ansible.com/ansible/latest/collections/ansible/windows/win_dsc_module.html.
 - [21] Proxmox Server Solutions. Proxmox virtual environment documentation, 2024. URL <https://pve.proxmox.com/pve-docs/>.
 - [22] Proxmox Server Solutions. Proxmox ve api viewer, 2024. URL <https://pve.proxmox.com/pve-docs/api-viewer/>.
 - [23] Telmate. Terraform provider for proxmox, 2023. URL <https://github.com/Telmate/terraform-provider-proxmox>.
-

-
- [24] BPG. Alternative terraform provider for proxmox ve, 2024. URL <https://github.com/bpg/terraform-provider-proxmox>.
 - [25] Cloudbase Solutions. Cloudbase-init documentation, 2023. URL <https://cloudbase-init.readthedocs.io/>.
 - [26] Microsoft Learn. Sysprep (system preparation) overview, 2022. URL <https://learn.microsoft.com/en-us/windows-hardware/manufacture/desktop/sysprep--system-preparation--overview>.
 - [27] Microsoft Learn. Windows remote management (winrm) service installation and configuration, 2023. URL <https://learn.microsoft.com/en-us/windows/win32/winrm/installation-and-configuration-for-windows-remote-management>.
 - [28] Microsoft Learn. Active directory domain services overview, 2023. URL <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/active-directory-domain-services>.
 - [29] Microsoft Learn. Dns server role overview, 2022. URL <https://learn.microsoft.com/en-us/windows-server/networking/dns/dns-top>.
 - [30] Microsoft Learn. File and storage services overview, 2023. URL <https://learn.microsoft.com/en-us/windows-server/storage/file-server/file-server-smb-overview>.
 - [31] Microsoft Learn. Access control overview (ntfs permissions), 2022. URL <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/manage/component-updates/access-control-overview>.
 - [32] Microsoft Learn. Microsoft smb protocol authentication and authorization overview, 2023. URL <https://learn.microsoft.com/en-us/windows-server/storage/file-server/smb-security>.
 - [33] Microsoft Learn. Enable access-based enumeration on a share, 2021. URL <https://learn.microsoft.com/en-us/windows-server/storage/dfs-namespaces/enable-access-based-enumeration-on-a-namespace>.
 - [34] Pester Foundation. Pester: Bdd testing framework for powershell, 2024. URL <https://pester.dev/>.
 - [35] Microsoft Learn. Test-smbshareaccess (storage cmdlets), 2022. URL <https://learn.microsoft.com/en-us/powershell/module/storage/test-smbshareaccess>.
-

- [36] Microsoft Learn. Active directory administration with powershell, 2023. URL <https://learn.microsoft.com/en-us/powershell/module/activedirectory/>.
 - [37] Microsoft Learn. Resolve-dnsname (dnsserver cmdlets), 2022. URL <https://learn.microsoft.com/en-us/powershell/module/dnsclient/resolve-dnsname>.
 - [38] Microsoft Learn. Test-netconnection (nettcpip cmdlets), 2023. URL <https://learn.microsoft.com/en-us/powershell/module/nettcpip/test-netconnection>.
 - [39] CCN-CERT. Guía ccn-stic-401: Directrices para la implantación del ens. Technical report, Centro Criptológico Nacional, 2022.
 - [40] CCN-CERT. Guía ccn-stic-801: Glosario de términos y abreviaturas del ens. Technical report, Centro Criptológico Nacional, 2022.
 - [41] CCN-CERT. Guía ccn-stic-570a25: Bastionado de microsoft windows server. Technical report, Centro Criptológico Nacional, 2025.
 - [42] CCN-CERT. Guía ccn-stic-573-25: Bastionado de servidor de ficheros. Technical report, Centro Criptológico Nacional, 2025.
 - [43] CCN-CERT. Guía ccn-stic-599ab23: Bastionado de sistemas windows cliente. Technical report, Centro Criptológico Nacional, 2023.
-

Lista de Acrónimos y Abreviaturas

ABE	Access-Based Enumeration.
AD	Active Directory.
AD DS	Active Directory Domain Services.
CCN	Centro Criptológico Nacional.
CERT	Computer Emergency Response Team.
DC	Domain Controller.
DNS	Domain Name System.
DSC	Desired State Configuration.
ENS	Esquema Nacional de Seguridad.
FS	File Server.
GPMC	Group Policy Management Console.
GPO	Group Policy Object.
IaC	Infrastructure as Code.
IIS	Internet Information Services.
ISO	Imagen de instalación en formato ISO.
LCM	Local Configuration Manager.
MOF	Managed Object Format.
NTFS	New Technology File System.
OU	Organizational Unit.
PKI	Public Key Infrastructure.
RDP	Remote Desktop Protocol.
SMB	Server Message Block.
SSH	Secure Shell.
TFG	Trabajo Fin de Grado.
VM	Virtual Machine.
WinRM	Windows Remote Management.
WSUS	Windows Server Update Services.

A. Código fuente y configuraciones de automatización

Este anexo recoge los principales fragmentos de código y configuraciones utilizados durante la implementación del flujo de automatización descrito en el Capítulo 4. El objetivo de incluirlos fuera del cuerpo principal del documento es mantener la explicación técnica centrada en las decisiones de diseño, la arquitectura y los resultados, sin perder la trazabilidad de la implementación realizada.

Los listados se agrupan según la fase del proceso a la que pertenecen: preparación de la imagen base, generación de la plantilla reutilizable, aprovisionamiento mediante Terraform, configuración declarativa mediante PowerShell DSC, aplicación de guías CCN-STIC y validación funcional del entorno.

A.1. Preparación de la imagen base

La preparación inicial del entorno parte de una ISO original de Windows Server, sobre la que se incorporan el archivo `autounattend.xml`, el script `Setup.ps1` y los componentes auxiliares necesarios para continuar el flujo de automatización.

A.1.1. Generación de la ISO personalizada

El Listado A.1 muestra el comando empleado para generar la ISO personalizada con soporte de arranque UEFI. Esta ISO se utiliza posteriormente para crear la primera máquina virtual base en Proxmox.

Código A.1: Generación de la ISO personalizada con `oscdimg`

```
1 set "SRC=C:\WS2025-UEFI"
2 set "DST=C:\WindowsServer2025Demo\WindowsServer2025-Autoun-uefi.iso"
3
4 set "EFI=%SRC%\efi\microsoft\boot\efisys_noprompt.bin"
5
6 oscdimg -m -o -u2 -udfver102 -h -lWS2025_AUTOUN ^
7 -bootdata:1#pEF,e,b"%EFI%" ^
8 "%SRC%" "%DST%"
```

A.1.2. Estructura funcional del script Setup.ps1

El script Setup.ps1 se ejecuta tras la instalación desatendida de Windows. Su función es preparar la máquina base para las fases posteriores: copia de módulos DSC, configuración de administración remota, instalación de Cloudbase-Init, integración con Proxmox y preparación del Local Configuration Manager.

El Listado A.2 muestra un fragmento de la estructura lógica del script, omitiendo detalles repetitivos o dependientes de rutas locales del laboratorio.

Código A.2: Estructura funcional del script Setup.ps1

```

1  Copy-DscModuleFromMedia -ModuleName 'NetworkingDsc' -MediaModulesRoot $mediaModules
2  Copy-DscModuleFromMedia -ModuleName 'ActiveDirectoryDsc' -MediaModulesRoot $mediaModules
3  Copy-DscModuleFromMedia -ModuleName 'DnsServerDsc' -MediaModulesRoot $mediaModules
4  Copy-DscModuleFromMedia -ModuleName 'ComputerManagementDsc' -MediaModulesRoot $mediaModules
5
6  # Configuración de red estática + PING(ICMP)
7  $ipAddress = "192.168.137.10" # IP VM
8  $prefixLen = 24 # /24 = 255.255.255.0
9  $gateway = "192.168.137.1" # Win10 host
10 $dnsServers = @("8.8.8.8")
11
12 # Cogemos el primer adaptador de red "normal" que este activo/levantado
13 $nic = Get-NetAdapter |
14     Where-Object { $_.Status -eq "Up" -and -not $_.Virtual } |
15     Select-Object -First 1
16
17 # IP estática + gateway
18 New-NetIPAddress -InterfaceIndex $nic.ifIndex -IPAddress $ipAddress `
19     -PrefixLength $prefixLen -DefaultGateway $gateway
20
21 # DNS
22 Set-DnsClientServerAddress -InterfaceIndex $nic.ifIndex -ServerAddresses $dnsServers
23
24 # Permitir ping
25 New-NetFirewallRule -DisplayName "Allow ICMPv4 In" -Protocol ICMPv4 `
26     -Direction Inbound -Action Allow -ErrorAction SilentlyContinue
27
28 # Preparación de administración remota y WinRM
29 winrm quickconfig -q
30 Enable-PSRemoting -Force -SkipNetworkProfileCheck
31 netsh advfirewall firewall add rule name="WinRM-HTTP" dir=in action=allow protocol=TCP ↔
32     ↔ localport=5985
33
34 # Instalación y habilitación de OpenSSH Server
35 Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0 -ErrorAction SilentlyContinue
36 Set-Service sshd -StartupType Automatic -ErrorAction SilentlyContinue
37 Start-Service sshd -ErrorAction SilentlyContinue
38 netsh advfirewall firewall add rule name="OpenSSH-Server-In-TCP" dir=in action=allow protocol=↔
39     ↔ TCP localport=22
40
41 # Instalación de Cloudbase-Init
42 $cloudbaseInstaller = Join-Path $filesRoot "CloudbaseInit\CloudbaseInitSetup.msi"
43 if (Test-Path $cloudbaseInstaller) {
44     Start-Process msiexec.exe -ArgumentList "/i `"$cloudbaseInstaller`" /qn" -Wait
45 }
46
47 # Copia de configuraciones de Cloudbase-Init

```

```

47 $cloudbaseConfSource = Join-Path $filesRoot "CloudbaseInit\conf"
48 $cloudbaseConfTarget = "C:\Program Files\Cloudbase Solutions\Cloudbase-Init\conf"
49
50 if (Test-Path $cloudbaseConfSource) {
51     Copy-Item -Path "$cloudbaseConfSource\*" -Destination $cloudbaseConfTarget -Recurse -Force
52 }
53
54 # Configuración básica del LCM
55 [DSCLocalConfigurationManager()]
56 Configuration LCMConfig {
57     Node localhost {
58         Settings {
59             RefreshMode = 'Push'
60             ConfigurationMode = 'ApplyOnly'
61             RebootNodeIfNeeded = $true
62         }
63     }
64 }
65
66 LCMConfig -OutputPath C:\DSC\LCM
67 Set-DscLocalConfigurationManager -Path C:\DSC\LCM -Force

```

A.2. Creación de la plantilla reutilizable

Una vez instalada y preparada la máquina base, se ejecuta Sysprep para eliminar información específica de la instalación original y dejar el sistema listo para ser convertido en plantilla de Proxmox.

A.2.1. Configuración base con 01-Base.ps1

El Listado A.3 muestra un fragmento del script utilizado para actualizar la máquina base y comprobar que está configurada antes de realizar Sysprep.

Código A.3: Actualización de la VM base

```

1
2 Service WinRMService {
3     Name = "WinRM"
4     StartupType = "Automatic"
5     State = "Running"
6 }
7
8 Script EnableRDPFirewall {
9     SetScript = {
10         Get-NetFirewallRule -Name "RemoteDesktop-*" -ErrorAction SilentlyContinue |
11         Enable-NetFirewallRule
12     }
13     TestScript = {
14         $r = Get-NetFirewallRule -Name "RemoteDesktop-*" -ErrorAction SilentlyContinue
15         if (-not $r) { return $false }
16         return (($r | Where-Object { $_.Enabled -eq 'False' } | Measure-Object).Count -eq 0)
17     }
18     GetScript = { @{ Result = "RDP rules state" } }
19     DependsOn = "[Service]WinRMService"
20 }
21
22 Script EnableWinRMFirewall {

```

```

23 SetScript = {
24     Get-NetFirewallRule -Name "WINRM-HTTP-In-TCP*" -ErrorAction SilentlyContinue |
25     Enable-NetFirewallRule
26 }
27 TestScript = {
28     $r = Get-NetFirewallRule -Name "WINRM-HTTP-In-TCP*" -ErrorAction SilentlyContinue
29     if (-not $r) { return $false }
30     return (($r | Where-Object { $_.Enabled -eq 'False' } | Measure-Object).Count -eq 0)
31 }
32 GetScript = { @{ Result = "WinRM rules state" } }
33 DependsOn = "[Service]WinRMService"
34 }
35
36 Script RegisterWindowsUpdateTask {
37     GetScript = { @{ Result = 'RegisterWindowsUpdateTask' } }
38
39     TestScript = { # Comprueba si existe ya la task }
40
41     SetScript = {
42         $ErrorActionPreference = 'Stop'
43
44         $taskName = 'TFG-Invoke-WindowsUpdate'
45         $script = 'C:\Provisioning\Invoke-WindowsUpdate.ps1'
46         $logPath = 'C:\Provisioning\Logs\Invoke-WindowsUpdate-launch.log'
47
48         if (-not (Test-Path $script)) {
49             throw "No existe $script"
50         }
51
52         $action = New-ScheduledTaskAction `
53             -Execute 'PowerShell.exe' `
54             -Argument "-NoProfile -ExecutionPolicy Bypass -File `"$script`" *> `"$logPath`""
55
56         $trigger = New-ScheduledTaskTrigger -AtStartup
57         $principal = New-ScheduledTaskPrincipal -UserId 'SYSTEM' -LogonType ServiceAccount -↔
58             ↳ RunLevel Highest
59
60         try {
61             Unregister-ScheduledTask -TaskName $taskName -Confirm:$false -ErrorAction ↳
62                 ↳ SilentlyContinue
63         } catch {}
64
65         Register-ScheduledTask `
66             -TaskName $taskName `
67             -Action $action `
68             -Trigger $trigger `
69             -Principal $principal `
70             -Description 'Ejecuta Windows Update para preparar la template'
71
72         DependsOn = "[Script]CopyWindowsUpdateScript"
73     }

```

A.2.2. Generalización con Sysprep y Cloudbase-Init

El Listado A.4 muestra el comando empleado para generalizar la máquina base. El archivo Unattend.xml pertenece a la configuración de Cloudbase-Init y permite preparar el primer arranque de los clones generados desde la plantilla.

Código A.4: Generalización de la máquina base con Sysprep y Cloudbase-Init


```
1 C:\Windows\System32\Sysprep\Sysprep.exe `
2 /generalize `
3 /oobe `
4 /shutdown `
5 /unattend:"C:\Program Files\Cloudbase Solutions\Cloudbase-Init\conf\Unattend.xml"
```

A.3. Aprovisionamiento con Terraform y Cloudbase-Init

Terraform se utiliza para desplegar los nodos finales del laboratorio desde la plantilla base generada previamente. La configuración define los recursos de cada máquina, sus parámetros de red y, en el caso del servidor de ficheros, un disco adicional para almacenamiento de datos.

A.3.1. Definición parametrizada de nodos (vars.tf)

El Listado A.5 muestra la variable utilizada dentro de `vars.tf` para describir los nodos principales del laboratorio.

Código A.5: Definición parametrizada de los nodos del laboratorio

```
1 variable "vms" {
2   description = "VMs a crear: vm_id => config"
3
4   type = map(object({
5     name = string
6     ip_cidr = string
7     gw = string
8     memory = number
9     cores = number
10    tags = list(string)
11  }))
12
13  default = {
14    100 = {
15      name = "DC1"
16      ip_cidr = "192.168.137.10/24"
17      gw = "192.168.137.1"
18      memory = 6144
19      cores = 3
20      tags = ["terraform", "ws2025", "dc"]
21    }
22
23    110 = {
24      name = "FS2"
25      ip_cidr = "192.168.137.11/24"
26      gw = "192.168.137.1"
27      memory = 5120
28      cores = 2
29      tags = ["terraform", "ws2025", "fs"]
30    }
31
32    120 = {
33      name = "CLI1"
34      ip_cidr = "192.168.137.12/24"
35      gw = "192.168.137.1"
36      memory = 5120
```

```
37     cores = 2
38     tags = ["terraform", "client", "ws2025"]
39   }
40 }
41 }
```

A.3.2. Clonado de máquinas desde la plantilla main.tf

El Listado A.6 contiene el código principal del fichero `main.tf`, utilizado para crear los clones desde la plantilla base. La inicialización se realiza mediante `configdrive2`, que posteriormente es leído por Cloudbase-Init dentro del sistema Windows.

Código A.6: Clonado de nodos desde la plantilla base mediante Terraform

```
1 resource "proxmox_virtual_environment_vm" "lab_nodes" {
2   for_each = var.vms
3
4   node_name = var.node_name
5   vm_id = tonumber(each.key)
6   name = each.value.name
7   tags = each.value.tags
8
9   clone {
10    vm_id = var.template_vm_id
11    full = true
12  }
13
14  cpu {
15    cores = each.value.cores
16    sockets = 1
17    type = "host"
18  }
19
20  memory {
21    dedicated = each.value.memory
22  }
23
24  initialization {
25    datastore_id = "local-lvm"
26    interface = "ide2"
27    type = "configdrive2"
28
29    ip_config {
30      ipv4 {
31        address = each.value.ip_cidr
32        gateway = each.value.gw
33      }
34    }
35
36    dns {
37      servers = ["192.168.137.10"]
38    }
39  }
40 }
```

A.3.3. Disco adicional para el servidor de ficheros (main.tf)

El Listado A.7 muestra la asignación condicional de un disco adicional al nodo que contiene la etiqueta `fs`. Este disco se utiliza posteriormente para almacenar los recursos compartidos del servidor de ficheros.

Código A.7: Asignación condicional de disco adicional al servidor de ficheros

```
1dynamic "disk" {
2  for_each = contains(each.value.tags, "fs") ? [1] : []
3
4  content {
5    datastore_id = var.disk_storage
6    interface = "scsi1"
7    size = 1
8
9    cache = "none"
10   discard = "on"
11   iothread = true
12   aio = "io_uring"
13   backup = true
14   ssd = true
15  }
16}
```

A.4. Configuración DSC del controlador de dominio

El controlador de dominio es el primer nodo configurado mediante DSC, ya que el resto del laboratorio depende de los servicios de Active Directory y DNS.

A.4.1. Promoción del controlador de dominio

El Listado A.8 recoge el fragmento principal de la configuración DSC encargada de instalar los roles necesarios, configurar DNS y crear el bosque de Active Directory.

Código A.8: Fragmento DSC para promoción del controlador de dominio

```
1WindowsFeature ADDS {
2  Name = 'AD-Domain-Services'
3  Ensure = 'Present'
4}
5
6WindowsFeature DNS {
7  Name = 'DNS'
8  Ensure = 'Present'
9}
10
11WindowsFeature RSAT_AD_PS {
12  Name = 'RSAT-AD-PowerShell'
13  Ensure = 'Present'
14}
15
16DnsServerAddress DnsToSelf {
17  Address = $DcIp
18  InterfaceAlias = $InterfaceAlias
19  AddressFamily = 'IPv4'
20}
```

```

20
21 DependsOn = @(
22     '[WindowsFeature]ADDS',
23     '[WindowsFeature]DNS',
24     '[WindowsFeature]RSAT_AD_PS'
25 )
26 }
27
28 ADDomain CreateForestRoot {
29     DomainName = $DomainName
30     DomainNetBiosName = $DomainNetbios
31     Credential = $BootstrapAdminCredential
32     SafemodeAdministratorPassword = $DsrmsCredential
33     ForestMode = 'WinThreshold'
34     DomainMode = 'WinThreshold'
35
36     DependsOn = '[DnsServerAddress]DnsToSelf'
37 }
38
39 DnsServerForwarder ForwardersConfig {
40     IsSingleInstance = 'Yes'
41     IPAddresses = $Forwarders
42
43     DependsOn = '[ADDomain]CreateForestRoot'
44 }

```

A.4.2. Creación de objetos de Active Directory

El Listado A.9 muestra la lógica utilizada para crear unidades organizativas, usuarios, grupos de seguridad y la cuenta BreakGlass de dominio.

Código A.9: Fragmento DSC para objetos de Active Directory

```

1 ADOrganizationalUnit OU_ServidoresDeFicheros {
2     Name = "ServidoresDeFicheros"
3     Path = $RootServidoresOU
4     Ensure = "Present"
5     ProtectedFromAccidentalDeletion = $true
6     Credential = $DomainAdminCredential
7
8     DependsOn = "[ADOrganizationalUnit]OU_Servidores"
9 }
10
11 ADUser UserIT {
12     UserName = "userIT"
13     Ensure = "Present"
14     DomainName = $DomainName
15     Password = $NormalUserPassword
16     Path = $UsersOU
17     Enabled = $true
18     Credential = $DomainAdminCredential
19     HomeDrive = $HomeDrive
20     HomeDirectory = "$HomeBase\userIT"
21
22     DependsOn = "[ADOrganizationalUnit]OU_Usuarios"
23 }
24
25 ADGroup GG_FS_IT_RW {
26     GroupName = "GG_FS_IT_RW"
27     GroupScope = "Global"
28     Category = "Security"

```

```

29 Path = $GroupsOU
30 Ensure = "Present"
31 MembersToInclude = @("userIT")
32 Credential = $DomainAdminCredential
33
34 DependsOn = @(
35     "[ADOrganizationalUnit]OU_Grupos",
36     "[ADUser]UserIT"
37 )
38 }
39
40 ADUser BG_DomainAdmin {
41     UserName = $BGDomainSam
42     Ensure = "Present"
43     DomainName = $DomainName
44     UserPrincipalName = $BGDomainUpn
45     Password = $BGDomainPassword
46     Path = $BreakGlassOU
47     Enabled = $true
48     PasswordNeverExpires = $true
49     Credential = $DomainAdminCredential
50
51     DependsOn = "[ADOrganizationalUnit]OU_BreakGlass"
52 }

```

A.5. Configuración DSC del servidor de ficheros

La configuración del servidor de ficheros se divide en dos fases: unión al dominio y preparación de recursos compartidos. Esta separación evita aplicar permisos basados en grupos de dominio antes de que el equipo pueda resolver correctamente dichos grupos.

A.5.1. Unión del servidor de ficheros al dominio

El Listado A.10 muestra la comprobación previa de DNS y la unión del servidor al dominio.

Código A.10: Fragmento DSC para unión del servidor de ficheros al dominio

```

1 Script WaitForDomainDns {
2     TestScript = {
3         try {
4             $name = "_ldap._tcp.dc._msdcs.$using:DomainName"
5             $dns = ($using:DnsServers | Select-Object -First 1)
6             $srv = Resolve-DnsName -Name $name -Type SRV -Server $dns -ErrorAction Stop
7             return [bool]$srv
8         }
9         catch {
10             return $false
11         }
12     }
13
14     SetScript = {
15         $name = "_ldap._tcp.dc._msdcs.$using:DomainName"
16         $dns = ($using:DnsServers | Select-Object -First 1)
17         $deadline = (Get-Date).AddMinutes(5)
18
19         while ((Get-Date) -lt $deadline) {

```

```

20     try {
21         $srv = Resolve-DnsName -Name $name -Type SRV -Server $dns -ErrorAction Stop
22         if ($srv) { return }
23     }
24     catch {}
25
26     Start-Sleep -Seconds 5
27 }
28
29 throw "Timeout esperando SRV: $name"
30 }
31
32 GetScript = {
33     @{ Result = "Domain SRV OK" }
34 }
35 }
36
37 Computer JoinDomain {
38     Name = $ComputerName
39     DomainName = $DomainName
40     Credential = $DomainJoinCredential
41     JoinOU = $JoinOU
42
43     DependsOn = "[Script]WaitForDomainDns"
44 }

```

A.5.2. Rol de ficheros, recurso SMB y Access-Based Enumeration

El Listado A.11 muestra la preparación del servidor de ficheros: instalación del rol, arranque del servicio SMB, preparación del volumen de datos, creación del recurso compartido y activación de Access-Based Enumeration.

Código A.11: Fragmento DSC para rol de ficheros, recurso SMB y ABE

```

1 WindowsFeature FileServerRole {
2     Name = "FS-FileServer"
3     Ensure = "Present"
4 }
5
6 Service LanmanServer {
7     Name = "LanmanServer"
8     StartupType = "Automatic"
9     State = "Running"
10
11     DependsOn = "[WindowsFeature]FileServerRole"
12 }
13
14 Script PrepareDataVolume {
15     TestScript = {
16         try {
17             $vol = Get-Volume -DriveLetter $using:DataDriveLetter -ErrorAction Stop
18             return ($vol.FileSystem -eq 'NTFS')
19         }
20         catch {
21             return $false
22         }
23     }
24
25     SetScript = {
26         $disk = Get-Disk |

```

```

27     Where-Object {
28         $_.IsSystem -eq $false -and $_.PartitionStyle -eq 'RAW'
29     } |
30     Select-Object -First 1
31
32     Initialize-Disk -Number $disk.Number -PartitionStyle GPT -PassThru | Out-Null
33
34     $part = New-Partition `
35         -DiskNumber $disk.Number `
36         -UseMaximumSize `
37         -DriveLetter $using:DataDriveLetter
38
39     Format-Volume `
40         -Partition $part `
41         -FileSystem NTFS `
42         -NewFileSystemLabel $using:VolumeLabel `
43         -Confirm:$false | Out-Null
44 }
45
46 GetScript = {
47     @{ Result = "Data volume checked/created" }
48 }
49
50 DependsOn = "[Service]LanmanServer"
51}
52
53File ShareRootFolder {
54     DestinationPath = $ShareRoot
55     Type = "Directory"
56     Ensure = "Present"
57
58     DependsOn = "[Script]PrepareDataVolume"
59}
60
61Script EnsureShares {
62     TestScript = {
63         $s = Get-SmbShare -Name $using:ShareName -ErrorAction SilentlyContinue
64         ($s -and $s.Path -eq $using:ShareRoot)
65     }
66
67     SetScript = {
68         if (-not (Get-SmbShare -Name $using:ShareName -ErrorAction SilentlyContinue)) {
69             New-SmbShare `
70                 -Name $using:ShareName `
71                 -Path $using:ShareRoot `
72                 -CachingMode None | Out-Null
73         }
74     }
75
76     GetScript = {
77         @{ Result = "Share ensured" }
78     }
79
80     DependsOn = "[File]ShareRootFolder"
81}
82
83Script ConfigureABE {
84     TestScript = {
85         (Get-SmbShare -Name $using:ShareName).FolderEnumerationMode -eq "AccessBased"
86     }
87
88     SetScript = {
89         Set-SmbShare `
90             -Name $using:ShareName `

```

```

91     -FolderEnumerationMode AccessBased `
92     -Force
93 }
94
95 GetScript = {
96     @{ Result = "ABE configured" }
97 }
98
99 DependsOn = "[Script]EnsureShares"
100}

```

A.5.3. Modelo de permisos NTFS y SMB

El Listado A.12 muestra una estructura representativa para aplicar permisos sobre carpetas del servidor de ficheros. La configuración concreta puede ampliarse con los grupos definidos en Active Directory para cada departamento o proyecto.

Código A.12: Estructura representativa para permisos NTFS y SMB

```

1Script ConfigureNtfsPermissions {
2    TestScript = {
3        # Comprobacion simplificada para el fragmento incluido en el anexo.
4        # En la ejecucion completa se valida el estado de las ACL antes de aplicar cambios.
5    }
6
7    SetScript = {
8        $path = $using:ShareRoot
9
10       icacls $path /inheritance:r
11
12       icacls $path /grant "SYSTEM:(OI)(CI)(F)"
13       icacls $path /grant "Administradores:(OI)(CI)(F)"
14       icacls $path /grant "ADTFG\GG_FS_IT_RW:(OI)(CI)(M)"
15       icacls $path /grant "ADTFG\GG_FS_IT_RO:(OI)(CI)(RX)"
16    }
17
18    GetScript = {
19        @{ Result = "NTFS permissions configured" }
20    }
21
22    DependsOn = "[Script]ConfigureABE"
23}
24
25Script ConfigureSmbPermissions {
26    TestScript = {
27        # En la version completa se comprueban los permisos SMB efectivos.
28        return $false
29    }
30
31    SetScript = {
32        Grant-SmbShareAccess `
33            -Name $using:ShareName `
34            -AccountName "ADTFG\GG_FS_IT_RW" `
35            -AccessRight Change `
36            -Force
37
38        Grant-SmbShareAccess `
39            -Name $using:ShareName `
40            -AccountName "ADTFG\GG_FS_IT_RO" `
41            -AccessRight Read `

```



```
42     -Force
43 }
44
45 GetScript = {
46     @{ Result = "SMB permissions configured" }
47 }
48
49 DependsOn = "[Script]ConfigureNtfsPermissions"
50 }
```

A.6. Configuración DSC del cliente Windows

El cliente Windows se utiliza como nodo de validación funcional. Su configuración incluye la preparación de una cuenta BreakGlass local, ajustes de red, unión al dominio y comprobación posterior de administración remota.

A.6.1. Preparación y unión del cliente al dominio

El Listado A.13 muestra el fragmento principal de configuración del cliente.

Código A.13: Fragmento DSC para preparación y unión del cliente Windows

```
1 Script BreakGlassLocal {
2     TestScript = {
3         try {
4             Get-LocalUser -Name $using:BGAdminLocalName -ErrorAction Stop | Out-Null
5
6             $isAdmin = Get-LocalGroupMember Administradores -ErrorAction SilentlyContinue |
7                 Where-Object {
8                     $_.Name -match "\\$( $using:BGAdminLocalName )"
9                 }
10
11             return [bool]$isAdmin
12         }
13         catch {
14             return $false
15         }
16     }
17
18     SetScript = {
19         $pw = ConvertTo-SecureString $using:BGAdminLocalPwd -AsPlainText -Force
20
21         if (-not (Get-LocalUser -Name $using:BGAdminLocalName -ErrorAction SilentlyContinue)) {
22             New-LocalUser `
23                 -Name $using:BGAdminLocalName `
24                 -Password $pw `
25                 -PasswordNeverExpires:$true `
26                 -AccountNeverExpires:$true
27         }
28
29         Add-LocalGroupMember `
30             -Group "Administradores" `
31             -Member $using:BGAdminLocalName `
32             -ErrorAction SilentlyContinue
33     }
34
35     GetScript = {
```

```
36 @{ Result = "BreakGlassLocal" }
37 }
38 }
39
40 NetAdapterBinding DisableIPv6 {
41     InterfaceAlias = $InterfaceAlias
42     ComponentId = 'ms_tcpip6'
43     State = 'Disabled'
44
45     DependsOn = "[Script]BreakGlassLocal"
46 }
47
48 Script EnsureDnsRegistrationEnabled {
49     SetScript = {
50         Set-DnsClient `
51             -InterfaceAlias $using:InterfaceAlias `
52             -RegisterThisConnectionsAddress $true `
53             -UseSuffixWhenRegistering $true `
54             -ConnectionSpecificSuffix $using:DomainName
55     }
56
57     TestScript = {
58         $c = Get-DnsClient -InterfaceAlias $using:InterfaceAlias
59
60         ($c.RegisterThisConnectionsAddress -eq $true) -and
61         ($c.UseSuffixWhenRegistering -eq $true)
62     }
63
64     GetScript = {
65         @{ Result = "DNS registration flags OK" }
66     }
67
68     DependsOn = "[NetAdapterBinding]DisableIPv6"
69 }
70
71 Computer JoinDomain {
72     Name = $ComputerName
73     DomainName = $DomainName
74     Credential = $DomainJoinCredential
75     JoinOU = $JoinOU
76
77     DependsOn = "[Script]EnsureDnsRegistrationEnabled"
78 }
79
80 Script GG_WS25_LocalAdmins_in_LocalAdministrators {
81     TestScript = {
82         $member = Get-LocalGroupMember -Group "Administradores" -ErrorAction SilentlyContinue |
83             Where-Object {
84                 $_.Name -eq "ADTFG\GG_WS25_LocalAdmins"
85             }
86
87         return [bool]$member
88     }
89
90     SetScript = {
91         Add-LocalGroupMember `
92             -Group "Administradores" `
93             -Member "ADTFG\GG_WS25_LocalAdmins" `
94             -ErrorAction Stop
95     }
96
97     GetScript = {
98         @{ Result = "Domain local admins ensured" }
99     }
```

```
100
101     DependsOn = "[Computer]JoinDomain"
102 }
```

A.7. Aplicación de guías CCN-STIC

La aplicación de guías CCN-STIC se realiza desde el controlador de dominio. Primero se copian los paquetes y scripts auxiliares al DC. Después se ejecuta una configuración DSC que importa las GPO y las enlaza a las unidades organizativas correspondientes.

A.7.1. Copia de paquetes y scripts al controlador de dominio

El Listado A.14 muestra el proceso de copia de paquetes ZIP y scripts auxiliares al controlador de dominio.

Código A.14: Copia de paquetes CCN-STIC y scripts auxiliares al controlador de dominio

```
1 $dc = "192.168.137.10"
2 $cred = Get-Credential "ADTFG\BG_DC_Domain"
3 $s = New-PSSession -ComputerName $dc -Credential $cred
4
5 Invoke-Command -Session $s -ScriptBlock {
6     $paths = @(
7         'Packages',
8         'Wrappers',
9         'State',
10        'Logs',
11        '_work',
12        'Extract'
13    ) | ForEach-Object {
14        "C:\CCN\$_"
15    }
16
17    New-Item -ItemType Directory -Path $paths -Force | Out-Null
18 }
19
20 # Copia de recursos mediante pipelines
21 @('570A25', '573-25', '599AB23') | ForEach-Object {
22     Copy-Item ".\CCN-STIC-$_-Scripts.zip" `
23         -Destination "C:\CCN\Packages\$_" `
24         -ToSession $s
25
26     Copy-Item ".\Apply-CCN$_.ps1" `
27         -Destination "C:\CCN\Wrappers\$_" `
28         -ToSession $s
29 }
30
31 Remove-PSSession $s
```

A.7.2. Configuración DSC para aplicar guías CCN-STIC

El Listado A.15 muestra la estructura de la configuración DSC utilizada para preparar el entorno de trabajo e invocar los scripts de aplicación de guías. Se muestra el

bloque de la guía 570A25; las guías 573-25 y 599AB23 siguen el mismo patrón.

Código A.15: Fragmento DSC para aplicar guías CCN-STIC de forma controlada

```
1 WindowsFeature GPMC {
2   Name = 'GPMC'
3   Ensure = 'Present'
4 }
5
6 Script AssertStaging {
7   PsDscRunAsCredential = $RunAsCred
8
9   TestScript = {
10     $ok = $true
11
12     foreach ($p in @( 'C:\CCN\Packages', 'C:\CCN\Wrappers' )) {
13       if (-not (Test-Path $p)) {
14         $ok = $false
15       }
16     }
17
18     return $ok
19   }
20
21   SetScript = {
22     New-Item `
23       -ItemType Directory `
24       -Path C:\CCN\Packages,C:\CCN\Wrappers,C:\CCN\State,C:\CCN\Logs,C:\CCN\_work,C:\CCN\Extract `
25       -Force | Out-Null
26   }
27
28   GetScript = {
29     @{ Result = 'Folders ensured' }
30   }
31
32   DependsOn = '[WindowsFeature]GPMC'
33 }
34
35 Script Apply570 {
36   PsDscRunAsCredential = $RunAsCred
37
38   TestScript = {
39     if (-not $using:Do570) {
40       return $true
41     }
42
43     if ($using:ForceReinstall570) {
44       return $false
45     }
46
47     return (Test-Path $using:State570)
48   }
49
50   SetScript = {
51     if (-not $using:Do570) {
52       return
53     }
54
55     & $using:Apply570 `
56       -ZipPath $using:Zip570 `
57       -Grade $using:Grade570 `
58       -Annexes $using:Annexes570 `
59       -ServersOuDn $using:ServersOuDn570 `
60       -ClientsOuDn $using:ClientsOuDn570 `
```

```

61     -LinkEnabled $using:LinkEnabled570
62 }
63
64 GetScript = {
65     @{ Result = '570 executed' }
66 }
67
68 DependsOn = '[Script]AssertStaging'
69 }

```

A.7.3. Ejecución de la aplicación de guías

El Listado A.16 muestra el comando utilizado para ejecutar la aplicación de guías CCN-STIC sobre el entorno del laboratorio.

Código A.16: Ejecución del script de aplicación de guías CCN-STIC

```

1 $cred = Get-Credential "ADTFG\BG_DC_Domain"
2
3 .\Invoke-CCN-Guides-DSC.ps1 `
4 -DcComputerName 192.168.137.10 `
5 -DomainCred $cred `
6 -Run570 -Run573 -Run599 `
7 -Grade570 ENS `
8 -Annexes570 @('A','B','C','D','E','F') `
9 -Nivel573 ESTANDAR `
10 -Nivel599 ESTANDAR `
11 -ClientsOuDn570 "OU=EstacionesWS25,OU=Clientes,DC=ad,DC=umtfg,DC=com" `
12 -ServersOuDn570 "OU=Servidores,DC=ad,DC=umtfg,DC=com" `
13 -TargetOuDns573 "OU=ServidoresDeFicheros,OU=Servidores,DC=ad,DC=umtfg,DC=com" `
14 -TargetOuDns599 "OU=EstacionesWS25,OU=Clientes,DC=ad,DC=umtfg,DC=com" `
15 -LinkEnabled570 Yes `
16 -LinkEnabled573 Yes `
17 -LinkEnabled599 Yes `
18 -PostCheck

```

A.8. Comandos de validación funcional

Además de las configuraciones de despliegue, se utilizaron comandos de comprobación para verificar que el entorno resultante cumplía los criterios de validación definidos en la metodología.

A.8.1. Comprobación del dominio y DNS

El Listado A.17 muestra los comandos utilizados para validar la disponibilidad del dominio y la resolución DNS interna.

Código A.17: Comprobación del dominio y resolución DNS

```

1 # Comprobar dominio desde el controlador de dominio
2 Get-ADDomain
3
4 # Comprobar registros SRV del dominio

```

```
5Resolve-DnsName `
6  -Name "_ldap._tcp.dc._msdcs.ad.umtfg.com" `
7  -Type SRV `
8  -Server 192.168.137.10
9
10# Comprobar resolucion del controlador de dominio
11Resolve-DnsName `
12  -Name "DC1.ad.umtfg.com" `
13  -Server 192.168.137.10
14
15# Comprobar resolucion del servidor de ficheros
16Resolve-DnsName `
17  -Name "FS2.ad.umtfg.com" `
18  -Server 192.168.137.10
```

A.8.2. Comprobación de unión al dominio

El Listado A.18 muestra los comandos utilizados para comprobar que los nodos se han incorporado correctamente a Active Directory.

Código A.18: Comprobación de unión de nodos al dominio

```
1# Buscar equipos registrados en Active Directory
2Get-ADComputer -Filter * |
3  Select-Object Name, DistinguishedName
4
5# Comprobar servidor de ficheros
6Get-ADComputer -Identity "FS2" |
7  Select-Object Name, DistinguishedName, Enabled
8
9# Comprobar cliente Windows
10Get-ADComputer -Identity "CLI1" |
11  Select-Object Name, DistinguishedName, Enabled
```

A.8.3. Comprobación del estado de DSC

El Listado A.19 muestra los comandos para revisar el estado de las configuraciones DSC aplicadas.

Código A.19: Comprobación del estado de DSC

```
1# Estado del Local Configuration Manager
2Get-DscLocalConfigurationManager
3
4# Ultimo estado de configuracion
5Get-DscConfigurationStatus |
6  Select-Object Status, StartDate, Type, RebootRequested
7
8# Historico de configuraciones aplicadas
9Get-DscConfigurationStatus -All |
10  Select-Object -First 5 Status, StartDate, Type, RebootRequested
```

A.8.4. Comprobación de recursos compartidos

El Listado A.20 muestra los comandos utilizados para validar la existencia del recurso compartido, la activación de ABE y los permisos efectivos.

Código A.20: Comprobación de recursos SMB y permisos

```
1# Comprobar recurso compartido
2Get-SmbShare -Name "TFGData"
3
4# Comprobar Access-Based Enumeration
5(Get-SmbShare -Name "TFGData").FolderEnumerationMode
6
7# Comprobar permisos SMB
8Get-SmbShareAccess -Name "TFGData"
9
10# Comprobar permisos NTFS
11icacls "F:\Shares\TFGData"
```

A.8.5. Comprobación de acceso desde cliente

El Listado A.21 muestra los comandos para comprobar el acceso permitido y denegado desde el cliente Windows unido al dominio.

Código A.21: Comprobación de acceso a recursos desde cliente Windows

```
1# Comprobar acceso al recurso compartido principal
2Test-Path "\\FS2\TFGData"
3
4# Comprobar acceso a carpeta del departamento IT
5Test-Path "\\FS2\TFGData\IT"
6
7# Comprobar acceso a carpeta restringida
8Test-Path "\\FS2\TFGData\RRHH"
9
10# Listar contenido visible para el usuario actual
11Get-ChildItem "\\FS2\TFGData"
```

A.8.6. Comprobación de administración remota

El Listado A.22 muestra los comandos utilizados para comprobar la administración remota antes y después del bastionado.

Código A.22: Comprobación de administración remota

```
1# Probar WinRM contra el controlador de dominio
2Test-WSMan 192.168.137.10
3
4# Probar WinRM contra el servidor de ficheros
5Test-WSMan 192.168.137.11
6
7# Crear sesion remota con cuenta BreakGlass de dominio
8$cred = Get-Credential "ADTFG\BG_DC_Domain"
9
10$s = New-PSSession `
11  -ComputerName 192.168.137.10 `
12  -Credential $cred
```

```
13
14 Invoke-Command -Session $s -ScriptBlock {
15     hostname
16     whoami
17 }
18
19 Remove-PSSession $s
```

A.9. Observaciones sobre credenciales y seguridad

Por motivos de seguridad y reproducibilidad, las contraseñas reales no deben incluirse en el documento. En los listados anteriores se muestran variables o llamadas a `Get-Credential` para representar el uso de credenciales durante la ejecución. En una implantación fuera del laboratorio, estas credenciales deberían integrarse en un mecanismo formal de gestión de secretos, con control de acceso, rotación y auditoría de uso.

En el contexto del laboratorio, las cuentas BreakGlass se utilizan como mecanismo de continuidad administrativa antes y después del bastionado, no como sustituto de un modelo formal de gestión de identidades en producción.
